

SAD

Presentan

Andres Henao Niño
Joan Emmanuel Orduz Chia
Joshua Prieto Zambrano
David Estevan Rodriguez Jurado

Director
Jaime Andrés Pavlich Mariscal

Pontificia Universidad Javeriana
Facultad de Ingeniería
Trabajo de Grado

Bogotá D.C.
mayo de 2026

Tabla de Contenidos

Tabla de Contenidos 2
Lista de Tablas 1
Lista de Figuras 1
1. Introducción 1
1.1 Propósito del documento 1
1.2 Descripción general del sistema 1
1.3 Objetivos de negocio que el sistema debe cumplir 1
1.4 Atributos de calidad principales 1
2. Visión general de los requisitos funcionales 1
3. Modelo de dominio 1
4. Stakeholders e intereses 1
5. Requisitos Arquitectónicamente Significativos (ASR) 1
6. Restricciones 1
7. Contexto y Alcance 1
7.1. Diagrama de Contexto del Sistema — Kit Pagos Colombia (Nivel 1) 1
7.1.1. Actores del sistema 1
7.1.2. Sistema principal 1
7.1.3. Sistemas externos 1
7.2 Diagrama de Contexto de la API de Simulación — Kit Pagos Colombia (Nivel 1) 1
7.2.1. Actor 1
7.2.2. Aplicación del Comercio (Huésped) 1
7.2.3. Kit Pagos Colombia 1
7.2.4. Pasarelas Reales (sistemas externos de referencia) 1
7.3. Alcance del sistema 1
8. Vista de contenedores 1
8.1 Diagrama de Contenedores — Kit Pagos Colombia (Nivel 2) 1
8.1.1. Contenedores del sistema 1
8.1.2. Relaciones entre contenedores 1
8.2 Diagrama de Contenedores — API de Simulación (Nivel 2) 1
8.2.1. Contenedores en el contexto de simulación 1
8.2.2. Relaciones entre contenedores en el flujo de simulación 1
9. Vista de componentes 1
9.1 Diagrama de Componentes — SDK Kit Pagos Colombia (Nivel 3) 1
9.1.1 Payment Facade 1
9.1.2. SDK Configurator 1
9.1.3. Gateway Factory 1
9.1.4. Payment Gateway Port 1

9.1.5. Wompi Adapter, Rapyd Adapter, Mercado Pago Adapter y Kushki Adapter..... 1
9.1.6. Response Normalizer..... 1
9.1.7. Webhook Verifier..... 1
9.1.8. Transaction Entity..... 1
9.1.9. Retry Handler..... 1
9.1.10. Error Handler..... 1
9.2 Diagrama de Componentes — API de Simulación (Nivel 3)..... 1
9.2.1. HTTP Router & Headers Middleware..... 1
9.2.2. Scenario Execution Engine..... 1
9.2.3. Gateway Mock Factory..... 1
9.2.4. Signature Generator..... 1
9.2.5. Webhook Trigger Endpoint..... 1
9.2.6. OpenAPI Documentation Provider..... 1
10. Vista de procesos..... 1
10.1 Diagramas de Secuencia — SDK..... 1
10.1.1 Creación de Pago (pendiente + webhook)..... 1
10.1.2 Pago Síncrono (Kushki — Tarjeta)..... 1
10.1.3 Conciliación por Webhook (asíncrono)..... 1
10.2 Diagramas de Secuencia — API de Simulación..... 1
10.2.1 Pago Exitoso (Escenario 1)..... 1
10.2.2 Pago Rechazado (Escenario 2)..... 1
10.2.3 Error de Red (Escenario 3)..... 1
10.2.4 Notificación Webhook (Escenario 4)..... 1
11. Vista física..... 1
11.1. Actores externos..... 1
11.1.1. Desarrollador / Tester QA..... 1
11.1.2. Aplicación del comercio..... 1
11.2. Infraestructura de despliegue..... 1
11.3. Relaciones y Flujos de Comunicación..... 1
12. Modelo de datos..... 1
13. Registros de Decisiones Arquitectónicas (ADR)..... 1
13.1. ADR-01: Adopción de Arquitectura Hexagonal para el núcleo del SDK..... 1
13.2. ADR-02: Adopción de Arquitectura Hexagonal para el núcleo del SDK..... 1
13.3. ADR-03: Adopción de Arquitectura Hexagonal para el núcleo del SDK..... 1
13.4. ADR-04: Verificación criptográfica de firmas para webhooks entrantes..... 1
13.5. ADR-05: Reintentos con backoff exponencial ante fallas transitorias..... 1
13.6. ADR-06: Motor de escenarios controlados en la API de Simulación..... 1
14. Riesgos técnicos..... 1
15. Estructura del Sistema..... 1

16. Glosario.....1
17. Referencias.....1

Lista de Tablas

Tabla 1. Requisitos funcionales..... 9
Tabla 2. Conceptos del Sistema..... 11
Tabla 3. Stakeholders e intereses..... 13
Tabla 4. Restricciones..... 15

Lista de Figuras

Figura 1. Diagrama de despliegue de la API de simulación..... 9
Figura 2. Diagrama de Contexto de Kit Pagos Colombia.....16
Figura 3. Diagrama de Contexto específico de la API de Simulación.....17
Figura 4. Diagrama de Contenedores de Kit Pagos Colombia..... 19
Figura 5. Diagrama de Contenedores de la API de Simulación..... 20
Figura 6. Diagrama de Componentes del SDK.....22
Figura 7. Diagrama de Componentes de la API de Simulación..... 25
Figura 8. Diagrama de Secuencia del SDK de creación de pago.....27
Figura 9. Diagrama de Secuencia del SDK de creación de pago síncrona.....28
Figura 10. Diagrama de Secuencia del SDK de conciliación por webhook..... 29
Figura 11. Diagrama de Secuencia de la API de Simulación de pago exitoso..... 30
Figura 12. Diagrama de Secuencia de la API de Simulación de pago rechazado..... 30
Figura 13. Diagrama de Secuencia de la API de Simulación de error de red.....31
Figura 14. Diagrama de Secuencia de la API de Simulación de Notificación webhook. 32

Figura 15. Diagrama de despliegue de la API de simulación..... 33
15.1 Núcleo del dominio.....41
15.2 SDK..... 41
15.3 API de Simulación..... 42

1. Introducción

1.1 Propósito del documento

Este documento corresponde a la Descripción de la Arquitectura de Software (SAD, por sus siglas en inglés) del proyecto Kit Pagos Colombia. Su propósito es presentar de forma estructurada las decisiones arquitectónicas que fundamentan el diseño y la construcción del sistema, describir los componentes que lo conforman, justificar las elecciones tecnológicas adoptadas y documentar los atributos de calidad que la arquitectura busca satisfacer.

El documento está dirigido principalmente a los miembros del equipo de desarrollo, al director del proyecto y a los evaluadores académicos del trabajo de grado. Un lector con conocimientos en ingeniería de software encontrará en este documento los elementos suficientes para comprender la estructura del sistema, razonar sobre sus decisiones de diseño y extender el framework con nuevas pasarelas sin necesidad de intervenir en su núcleo.

1.2 Descripción general del sistema

Kit Pagos Colombia es un framework de código abierto implementado en TypeScript sobre Node.js 18 LTS, cuyo propósito es simplificar el proceso de integración de múltiples pasarelas de pago en aplicaciones de software desarrolladas en el contexto colombiano. El ecosistema de pagos digitales en Colombia está actualmente fragmentado entre cuatro proveedores principales (Wompi, Rapyd, Mercado Pago y Kushki), cada uno con sus propias convenciones de autenticación, formatos de respuesta, mecanismos de notificación asíncrona y comportamientos ante errores. Esta fragmentación obliga a los equipos de desarrollo a reimplementar lógica específica por cada proveedor, multiplicando el esfuerzo, la probabilidad de errores y el tiempo invertido cada vez que un proyecto requiere incorporar una pasarela adicional.

El framework aborda este problema mediante tres componentes complementarios. El primero es un SDK con interfaz unificada que abstrae las diferencias entre proveedores y expone un conjunto consistente de operaciones (creación de pagos, consulta de estado y validación de webhooks) independientemente de la pasarela activa. El segundo es una API web de simulación desplegada que replica el comportamiento de las pasarelas para los escenarios de prueba más frecuentes, sin requerir credenciales reales ni configuración local adicional. El tercero es una documentación centralizada de datos de prueba que consolida, por pasarela, las tarjetas de prueba, credenciales de sandbox y referencias de pago disponibles en cada proveedor.

1.3 Objetivos de negocio que el sistema debe cumplir

La arquitectura del framework está orientada a satisfacer los siguientes objetivos, derivados directamente de los objetivos específicos del proyecto de grado:

- El framework debe permitir que un desarrollador integre cualquiera de las cuatro pasarelas soportadas utilizando la misma interfaz de programación, de modo que cambiar de proveedor implique únicamente modificar un parámetro de configuración sin alterar el código de negocio de la aplicación.
- El framework debe ofrecer un entorno de pruebas controlado a través de la API de simulación, que permita reproducir escenarios de éxito, rechazo, error de red y notificaciones asíncronas sin depender de la disponibilidad ni de las restricciones de los sandboxes oficiales de cada proveedor.
- El framework debe ser extensible, de modo que la incorporación de una nueva pasarela en el futuro requiera únicamente la implementación de un nuevo adaptador sin modificar el núcleo del sistema.
- El framework debe ser evaluable mediante métricas objetivas de calidad de código. Su impacto se medirá comparando los valores de WMC, CBO y RFC, métricas del conjunto propuesto por Chidamber y Kemerer (1994) entre una integración directa con las APIs nativas de las pasarelas y una integración equivalente utilizando el framework.

1.4 Atributos de calidad principales

Los atributos de calidad que la arquitectura debe satisfacer se derivan de los requisitos no funcionales especificados en el documento SRS del proyecto, organizados bajo la clasificación FURPS+:

Extensibilidad. La arquitectura hexagonal (Ports & Adapters) garantiza que el núcleo del SDK sea independiente de cualquier pasarela específica. La incorporación de un nuevo proveedor requiere únicamente implementar una clase adaptadora que cumpla el contrato del puerto, sin modificar ningún componente existente.

Usabilidad. El framework debe poder instalarse mediante un único comando y su documentación debe permitir que un desarrollador con conocimientos básicos de Node.js complete su primera integración en un tiempo máximo de dos horas, verificado mediante una sesión de prueba de usabilidad durante la fase de evaluación.

Confiabilidad. El SDK no debe lanzar excepciones no manejadas bajo ninguna condición de error esperada. Ante errores transitorios implementa reintentos automáticos con backoff exponencial (3 intentos con intervalos de 1s, 2s y 4s por defecto). La API de simulación debe permanecer accesible durante todo el período de evaluación del proyecto.

Rendimiento. El overhead introducido por el framework frente a una llamada directa a la API de cada pasarela no debe superar los 100 milisegundos en condiciones de red normales. La API de simulación debe responder en menos de 500 milisegundos para escenarios sin latencia intencionada.

Seguridad. Toda comunicación entre el SDK y las APIs de las pasarelas se realiza exclusivamente a través de HTTPS. Las credenciales de cada proveedor no deben ser expuestas en logs, mensajes de error ni en las respuestas del SDK. El framework no almacena ni procesa datos de tarjeta de crédito directamente, esa responsabilidad recae en los mecanismos certificados de cada pasarela.

Mantenibilidad. El código del framework debe seguir las convenciones de estilo definidas en el repositorio, verificables mediante el comando ***npm run lint***. Cada adaptador debe estar acompañado de pruebas de integración que verifiquen su funcionamiento contra la API de simulación.

2. Visión general de los requisitos funcionales

En la tabla 1 se presenta el listado consolidado de requisitos funcionales del framework, organizados por componente. Las prioridades se clasifican en Alta (esencial para el MVP), Media (importante pero no bloqueante) y Baja (deseable en versiones futuras).

ID	Descripción del requisito	Componente	Prioridad
RF-01	El sistema debe permitir crear una intención de pago especificando pasarela, monto, moneda, referencia de orden y datos del pagador mediante una interfaz unificada.	SDK	Alta
RF-02	El sistema debe retornar una respuesta normalizada ante la creación de un pago, incluyendo el identificador de transacción, el estado inicial y la URL o token de pago según la pasarela.	SDK	Alta
RF-03	El sistema debe permitir consultar el estado de una transacción por identificador y retornarlo mapeado a los estados normalizados: PENDIENTE, APROBADO, RECHAZADO, EXPIRADO o ERROR.	SDK	Alta
RF-04	El sistema debe validar la firma criptográfica de un webhook entrante utilizando el algoritmo correspondiente a cada pasarela y retornar un evento normalizado si la firma es válida.	SDK	Alta
RF-05	El sistema debe lanzar excepciones tipadas con código de error normalizado y mensaje descriptivo ante cualquier condición de fallo esperada (timeout, credenciales inválidas, recurso no encontrado).	SDK	Alta
RF-06	El sistema debe permitir cambiar la pasarela activa modificando únicamente el parámetro de configuración en la inicialización del framework, sin modificar el código de negocio del llamador.	SDK	Alta
RF-07	El sistema debe reintentar automáticamente una operación ante errores transitorios (timeout, respuestas 5xx) siguiendo una estrategia de backoff exponencial configurable, hasta un número máximo de intentos definido por el usuario.	SDK	Media
RF-08	El sistema debe garantizar que las credenciales de cada pasarela no sean expuestas en logs, mensajes de error ni en las respuestas del SDK.	SDK	Alta
RF-09	El sistema debe exponer endpoints REST que repliquen el comportamiento de creación de pago de cada pasarela soportada, accesibles sin credenciales reales.	API Simulación	Alta
RF-10	El sistema debe permitir configurar el escenario de respuesta de cada endpoint mediante un parámetro de la solicitud,	API Simulación	Alta

	soportando al menos: APROBADO, RECHAZADO, FONDOS_INSUFICIENTES, TIMEOUT y ERROR_RED.		
RF-11	El sistema debe replicar la estructura exacta de las respuestas de error de cada pasarela en los escenarios de fallo configurados.	API Simulación	Alta
RF-12	El sistema debe permitir solicitar el envío de un webhook simulado hacia una URL destino, con el formato y firma correspondientes a la pasarela indicada.	API Simulación	Media
RF-13	El sistema debe exponer la especificación OpenAPI 3.0 de la API de simulación en un endpoint público (/docs), accesible a través de un navegador web.	API Simulación	Media
RF-14	El sistema debe proveer documentación de datos de prueba organizada por pasarela, incluyendo tarjetas de prueba, credenciales de sandbox y referencias de pago válidas para cada proveedor.	Documentación	Alta
RF-15	El sistema debe indicar el comportamiento esperado de cada dato de prueba (tarjeta aprobada, tarjeta rechazada, fondos insuficientes) junto con la fuente oficial del proveedor.	Documentación	Media

Tabla 1. Requisitos funcionales

3. Modelos de dominio

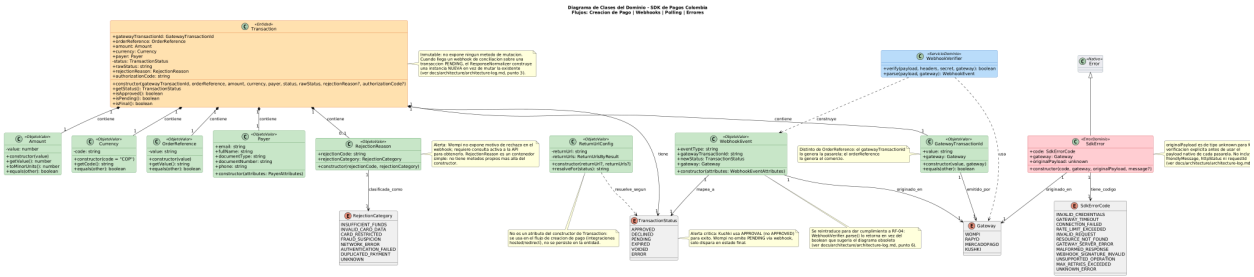


Figura 1. Diagrama de clases del dominio

Concepto	Explicación

Transaction (Entidad)	Representa una transacción de pago de principio a fin: agrupa el monto, la moneda, la referencia del comercio y los datos del pagador, y una vez que la pasarela responde, también el identificador nativo y el estado. Es la única clase con identidad propia; todas las demás son objetos de valor sin identidad.
Amount (Objeto de Valor)	Encapsula el monto en unidades monetarias con hasta dos decimales — la representación canónica del SDK. Cada adaptador es responsable de convertirla al formato nativo de su pasarela (centavos, objeto descompuesto, etc.).
Currency (Objeto de Valor)	Encapsula el código ISO 4217 de la moneda (por defecto "COP") y valida que tenga exactamente tres letras mayúsculas.
OrderReference (Objeto de Valor)	Es la referencia que el comercio asigna a la transacción para correlacionarla con sus propios sistemas. El SDK valida que no esté vacía, pero nunca la genera ni garantiza su unicidad — eso es responsabilidad del comercio.
Payer (Objeto de Valor)	Agrupa los datos de quien paga (email, nombre, tipo y número de documento, teléfono). El único campo obligatorio es el email, por ser el único dato que las cuatro pasarelas exigen en común.
ReturnUrlConfig (Objeto de Valor)	Configura a dónde se redirige al usuario después de pagar. Solo aplica en integraciones hosted/redirect. Contempla una URL única o URLs diferenciadas por resultado (success/failure/pending), porque Mercado Pago es la única pasarela con esa granularidad.
GatewayTransactionId (Objeto de Valor)	El identificador que la pasarela genera internamente para la transacción — distinto de OrderReference. Se usa para consultas de estado, reembolsos y conciliación contra la pasarela.
RejectionReason (Objeto de Valor)	Explica por qué se rechazó una transacción, combinando el código nativo de la pasarela (rejectionCode) con una categoría semántica normalizada (rejectionCategory) que el comercio puede usar para decidir su lógica de negocio.
WebhookVerifier (Servicio de Dominio)	Verifica que un evento de webhook realmente venga de la pasarela y no haya sido alterado ni falsificado. Se modela como servicio, no como entidad ni objeto de valor, porque no tiene datos propios que persistir — es comportamiento puro.

SdkError (Error de Dominio)	El único tipo de error que el SDK expone hacia afuera, sin importar qué pasarela lo originó. Extiende el Error nativo y guarda el payload original sin transformarlo como unknown, para forzar una verificación explícita antes de usarlo.
TransactionStatus (Enumeración)	Los seis estados posibles de una transacción, ya normalizados entre las cuatro pasarelas: **APPROVED, DECLINED, PENDING, EXPIRED, VOIDED, ERRO**R. Es el único vocabulario de estado que el código del comercio debe conocer.
RejectionCategory (Enumeración)	Clasifica el motivo de un rechazo en ocho categorías de negocio (fondos insuficientes, datos inválidos, tarjeta restringida, fraude, error de red, autenticación fallida, pago duplicado, desconocido), para que la lógica de reintento no dependa de interpretar strings distintos por pasarela.
SdkErrorCode (Enumeración)	Códigos de error técnico normalizados (timeout, credenciales inválidas, límite de solicitudes, etc.), independientes de qué pasarela los causó. Cada uno tiene un mensaje amigable asociado en el catálogo centralizado del SDK.
Gateway (Enumeración)	Identifica cuál de las cuatro pasarelas está involucrada (WOMPI, RAPYD, MERCADOPAGO, KUSHKI). Se usa tanto en GatewayTransactionId como en SdkError para saber de dónde vino cada dato.
SimulationRequestHeaders (Objeto de Valor)	Forma del resultado que produce HttpRouterMiddleware.extractHeaders(req): encapsula scenario (SimulationScenario) y delayMs (número opcional, solo relevante para TIMEOUT). Es el objeto que ScenarioExecutionEngine.execute() recibe como entrada.
MockPaymentResult (Objeto de Valor)	Representación interna normalizada que ScenarioExecutionEngine construye para los tres escenarios de negocio (APPROVED, DECLINED, INSUFFICIENT_FUNDS) antes de que GatewayMockFactory la traduzca a la forma nativa exacta de cada pasarela. Encapsula gatewayTransactionId, gateway, scenario, amount, currency, rawStatus (el valor nativo que la pasarela real emitiría) y, cuando aplica, un rejectionReason. No se usa para TIMEOUT ni NETWORK_ERROR, que no delegan en GatewayMockFactory (9.2.2).

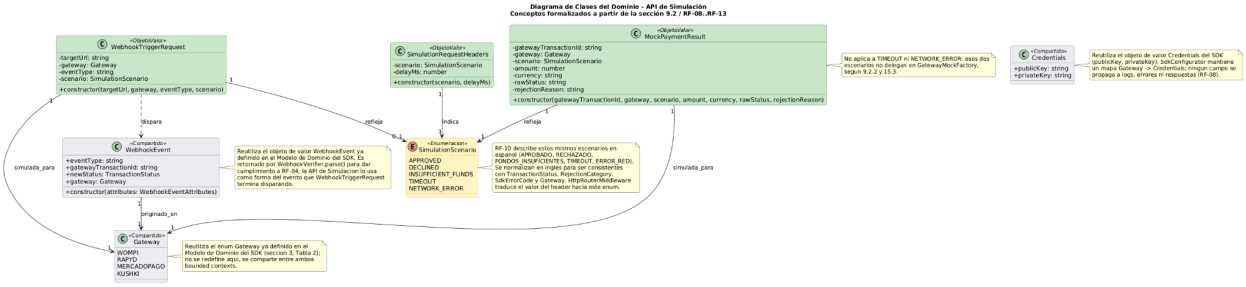


Figura 2. Diagrama de despliegue de la API de simulación

Concepto	Explicación
SimulationScenario (Enumeración)	Los cinco escenarios controlables que un desarrollador o tester puede solicitar mediante el header x-simulate-scenario, normalizados en inglés para ser consistentes con el resto del vocabulario del dominio: APPROVED, DECLINED, INSUFFICIENT_FUNDS, TIMEOUT, NETWORK_ERROR. RF-10 los describe en español (APROBADO, RECHAZADO, FONDOS_INSUFICIENTES, TIMEOUT, ERROR_RED); son equivalentes semánticos — HttpRouterMiddleware traduce el valor del header hacia este enum interno.
WebhookTriggerRequest (Objeto de Valor)	Forma de la solicitud que WebhookTriggerEndpoint recibe en POST /v1/sim/webhooks/trigger. Encapsula targetUrl, gateway, eventType y scenario (el resultado que el webhook simulado debe reflejar). SignatureGenerator y GatewayMockFactory la usan para construir el payload firmado que finalmente se despacha (RF-12).
WebhookEvent (Objeto de Valor)	(Objeto de Valor)Representa el evento notificado de forma asíncrona por una pasarela cuando cambia el estado de una transacción. Encapsula el identificador nativo de la transacción (gatewayTransactionId), el tipo de evento normalizado, la pasarela de origen (gateway), el estado resultante mapeado al dominio (status) y el payload nativo original (rawPayload). Es retornado por WebhookVerifier.parse() para dar cumplimiento a RF-04, en lugar del boolean que el diagrama obsoleto sugería.
Credentials (Objeto de Valor)	Encapsula el par de llaves (publicKey, privateKey) que el desarrollador configura por pasarela mediante SdkConfigurator.configure(options). SdkConfigurator mantiene un mapa Gateway → Credentials y cada Adapter las usa internamente para autenticarse ante su proveedor, sin que se propaguen a logs, errores ni respuestas del SDK (RF-08).

4. Stakeholders e intereses

Una tabla que especifique cada stakeholder, su información de contacto y qué intereses o expectativas tiene sobre el sistema que se está desarrollando (cómo espera beneficiarse del mismo).

Rol	Información de contacto	Intereses / expectativas
Autor(es) del proyecto	*Andrés Henao Niño :* ahenaon@javeriana.edu.co *Joshua Prieto Zambrano :*	Cumplir los objetivos de negocio de la sección 1.3 — interfaz unificada entre pasarelas, extensibilidad, entorno de pruebas controlado y mejores métricas de calidad de código (WMC, CBO, RFC) frente a una integración directa — usando la arquitectura hexagonal como decisión

	prietojoshua@javeriana.edu.co *Joan Emmanuel Orduz Chia : joan_orduz@javeriana.edu.co* *David Estevan Rodriguez Jurado : rodriguezj.david@javeriana.edu.co*	de diseño (ADR-01, sección 13.1). Entregar funcionales el SDK, la API de Simulación y la documentación centralizada dentro del tiempo disponible.
Director del proyecto de grado / Asesor	*Jaime Andres Pavlich Mariscal : jpavlich@javeriana.edu.co*	Coherencia con el SRS y el SPMP; seguimiento del avance frente al cronograma.
Evaluadores académicos / Jurado de tesis	*Se asigna en la sustentación*	Rigor en la comparación de métricas CK; alcance delimitado con claridad (sección 7.3).
Desarrollador de software / Tester QA (usuarios finales)	*No se define un canal de contacto formal en el alcance del proyecto.*	Integración ágil (instalación y primeros pagos en <2h, sección 1.4); conmutación de pasarela por configuración (sección 1.3); pruebas sin credenciales reales mediante simulación (RF-09, RF-10); errores tipados y recuperación automática ante fallas (ADR-04, ADR-05).

Tabla 3. Stakeholders e intereses

5. Requisitos Arquitectónicamente Significativos (ASR)

Los Requisitos Arquitectónicamente Significativos de Kit Pagos Colombia se derivaron cruzando los atributos de calidad que motivaron las decisiones de diseño ya documentadas (arquitectura hexagonal, normalización de respuestas, manejo de reintentos y verificación de firmas) con los requisitos funcionales definidos para el SDK y la API de Simulación. La tabla siguiente organiza estos requisitos en forma de árbol de utilidad, relacionando cada atributo de calidad con su sub-atributo específico, las funcionalidades que lo materializan y un escenario concreto que permite verificarlo. Los ASR están ordenados de mayor a menor prioridad, comenzando por aquellos que sustentan la propuesta central de la tesis (la abstracción unificada de múltiples pasarelas) y terminando por aquellos de carácter complementario.

Con el fin de que el texto no quede comprimido ni se obligue a reducir el tamaño de la fuente para el entendimiento de la tabla, se optó por adjuntar una hoja de cálculo con el contenido de esta sección: Requisitos Arquitectónicamente Significativos (ASR) - Kit Pagos Colombia .

6. Restricciones

Indique cualquier restricción que limite la libertad de los ingenieros a la hora de decidir la arquitectura del sistema y las soluciones a los ASR. Las principales categorías de restricciones son:

- Técnicas: Directamente relacionadas con las tecnologías, métodos de ingeniería, sistemas de software, etc.
- Organizacionales/políticas: Situaciones internas o externas a la organización y de las personas que limiten las decisiones arquitectónicas del sistema
- Convencionales: Necesidades de seguimiento de estándares, guías de codificación, leyes, etc.

Restricción	Categoría	Descripción
Lenguaje único del SDK	Técnica	El SDK se implementa exclusivamente en TypeScript/Node.js 18 LTS en adelante. Esto descarta cualquier optra decisión arquitectónica que dependa de otro runtime o lenguaje (por ejemplo no se puede resolver el cálculo de metricas CK con herramientas Java estándar, se debe usar ts-morph)
Dependencia de sandboxes externos	Técnica	El framework depende de la disponibilidad de los ambientes sandbox de cada pasarela para su validación. Cambios en las APIs de los proveedores pueden requerir actualizaciones en los adaptadores, lo que introduce una dependencia externa no controlable por el equipo.
Infraestructura de nivel gratuito	Organizacional/política	El despliegue de la API de Simulación se realiza sobre una plataforma PaaS de plan gratuito o bajo costo (Render), lo que impone límites de disponibilidad, rendimiento y tiempos de arranque en frío que la arquitectura debe tolerar.
Duración del proyecto académico	Organizacional/política	El proyecto se desarrolla en un período de cinco meses (julio–noviembre 2026) como trabajo de grado, lo que limita la cantidad de pasarelas integradas, la profundidad de cada adaptador y descarta decisiones arquitectónicas que requieran más tiempo de maduración.
Cumplimiento de PCI DSS por delegación	Convencional	El framework no puede procesar ni almacenar datos de tarjeta de crédito directamente; toda operación sensible

		debe delegarse a los mecanismos certificados de cada pasarela, en cumplimiento del estándar PCI DSS.
Términos de uso de las APIs de los proveedores	Convencional	El uso en producción está sujeto a los términos y condiciones de las APIs de Wompi, Rapyd, Mercado Pago y Kushki; el framework no puede modificar ni interceptar los flujos de autenticación internos de ninguna pasarela.

Tabla 4. Restricciones

7. Contexto y Alcance

7.1. Diagrama de Contexto del Sistema — Kit Pagos Colombia (Nivel 1)

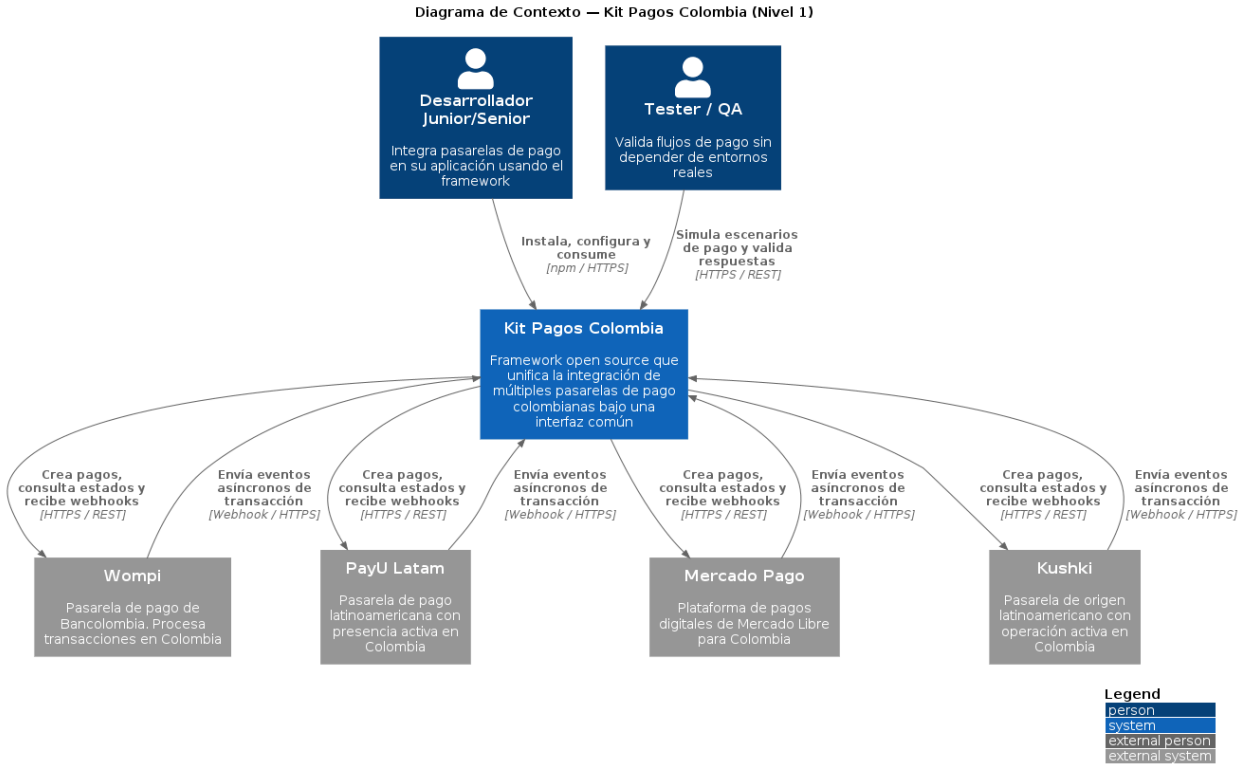


Figura 2. Diagrama de Contexto de Kit Pagos Colombia

El diagrama de contexto del sistema muestra la posición de Kit Pagos Colombia dentro de su entorno, identificando los actores que interactúan con él y los sistemas externos con los que se comunica.

7.1.1. Actores del sistema

El sistema tiene dos tipos de usuario. El primero es el **Desarrollador Junior o Senior**, quien instala el framework en su entorno de trabajo mediante npm, lo configura con las credenciales de las pasarelas que desea utilizar y consume su interfaz unificada para integrar capacidades de pago en su aplicación. El segundo es el **Tester o QA**, quien interactúa con el framework principalmente a través de la API de simulación para validar flujos de pago en escenarios controlados sin depender de entornos reales ni de credenciales de producción.

7.1.2. Sistema principal

Kit Pagos Colombia actúa como intermediario técnico entre las aplicaciones de software y las pasarelas de pago colombianas. Su propósito es abstraer la heterogeneidad del ecosistema de proveedores, exponiéndose como una interfaz común, predecible y reutilizable. El sistema no procesa pagos directamente (delega cada operación hacia el proveedor seleccionado) sino que normaliza la comunicación, los formatos de respuesta y el manejo de errores para que la aplicación consumidora no tenga que conocer las convenciones de ningún proveedor en particular.

7.1.3. Sistemas externos

El sistema se comunica con cuatro pasarelas de pago, cada una operando como sistema externo independiente:

Wompi es la pasarela de Bancolombia, con mayor penetración en el mercado colombiano. El framework envía solicitudes de creación de pago y consulta de estado a su API REST mediante HTTPS, y recibe notificaciones asíncronas de cambio de estado a través de webhooks firmados con SHA-256.

Rapyd Latam es la pasarela que sustituye a PayU Latam en el ecosistema de Kit Pagos Colombia, tras la adquisición de su operación en Latinoamérica en 2025. A diferencia de las otras tres pasarelas, Rapyd no recibe las credenciales dentro del cuerpo de la solicitud sino en los encabezados HTTP. Cada petición debe llevar un access_key propio del comercio y una firma calculada con HMAC-SHA256, que se construye concatenando el método HTTP, la ruta del recurso al que se llama, una cadena aleatoria (salt), la marca de tiempo de la petición, el access_key, el secret_key y el cuerpo del mensaje, y luego codificando ese resultado en Base64. Para las notificaciones asíncronas la fórmula es prácticamente la misma, con dos diferencias: no incluye el método HTTP, y en vez de la ruta relativa usa la URL pública completa que el comercio configuró para recibir los webhooks. Esta segunda fórmula es importante porque implica que el adaptador necesita conocer de antemano esa URL como un dato de configuración, algo que ninguna de las otras tres pasarelas exige.

Mercado Pago es la plataforma de pagos de Mercado Libre para Colombia. La comunicación se realiza mediante autenticación Bearer y sus webhooks utilizan HMAC-SHA256.

Kushki es una pasarela de origen latinoamericano con operación activa en Colombia. La autenticación se realiza mediante el header ***Private-Merchant-Id*** y sus notificaciones asíncronas también usan HMAC-SHA256.

En todos los casos, la relación bidireccional refleja dos flujos distintos: las solicitudes salientes del framework hacia cada pasarela (creación de pago, consulta de estado) y los eventos entrantes que cada pasarela envía hacia el framework mediante webhooks cuando el estado de una transacción cambia.

7.2 Diagrama de Contexto de la API de Simulación — Kit Pagos Colombia (Nivel 1)

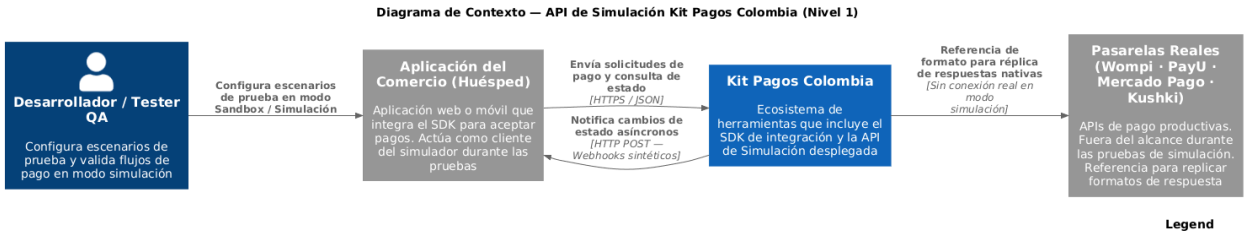


Figura 3. Diagrama de Contexto específico de la API de Simulación

Este diagrama complementa el anterior con un enfoque específico en el flujo de simulación, mostrando cómo interactúan los actores, la aplicación del comercio, el framework y las pasarelas reales durante las pruebas de desarrollo.

7.2.1. Actor

En el contexto de simulación, el **Desarrollador y el Tester QA** comparten el mismo rol: configurar escenarios de prueba en modo sandbox o simulación y validar que la integración responde correctamente ante cada uno de ellos.

La distinción entre ambos perfiles se diluye en este flujo porque el objetivo es el mismo, verificar el comportamiento del sistema sin transacciones reales.

7.2.2. Aplicación del Comercio (Huésped)

La aplicación del comercio es el sistema externo que integra el SDK de Kit Pagos Colombia para aceptar pagos. En el contexto de simulación actúa como cliente del simulador: envía solicitudes de pago y consulta de estado hacia el framework mediante HTTPS y JSON, y recibe notificaciones asíncronas de cambio de estado a través de webhooks sintéticos generados por la API de simulación mediante HTTP POST.

Este elemento es relevante en el diagrama porque evidencia que el framework no es el destino final de las solicitudes, es un intermediario que procesa y redirige, y cuyo comportamiento debe ser transparente para la aplicación que lo consume.

7.2.3. Kit Pagos Colombia

En este diagrama el sistema aparece en su rol de simulador: recibe las solicitudes de la aplicación del comercio, las procesa internamente según el escenario configurado y retorna respuestas que replican el comportamiento de las pasarelas reales sin establecer conexión efectiva con ninguna de ellas.

7.2.4. Pasarelas Reales (sistemas externos de referencia)

En el contexto de simulación, las pasarelas reales (Wompi, Rapyd, Mercado Pago y Kushki) aparecen fuera del alcance del flujo activo. No se establece ninguna conexión real con ellas durante las pruebas de simulación. Su presencia en el diagrama cumple un propósito documental: son la referencia a partir de la cual la API de simulación construye sus respuestas, replicando con exactitud los formatos, los campos y los valores que cada proveedor retornaría en un entorno productivo real.

7.3. Alcance del sistema

El sistema abarca exclusivamente la capa de integración entre la aplicación del comercio y las pasarelas de pago. No forma parte de su alcance el procesamiento directo de datos de tarjeta de crédito, la gestión de cuentas de comercio en cada proveedor, la conciliación financiera de transacciones, ni ninguna operación administrativa sobre las APIs de los proveedores. La responsabilidad sobre el cumplimiento de PCI DSS recae en las pasarelas integradas y en las aplicaciones que consumen el framework, no en el framework mismo, que actúa exclusivamente como intermediario que delega cada operación hacia el proveedor correspondiente sin interceptar ni almacenar datos sensibles del proceso de pago.

8. Vista de contenedores

8.1 Diagrama de Contenedores — Kit Pagos Colombia (Nivel 2)

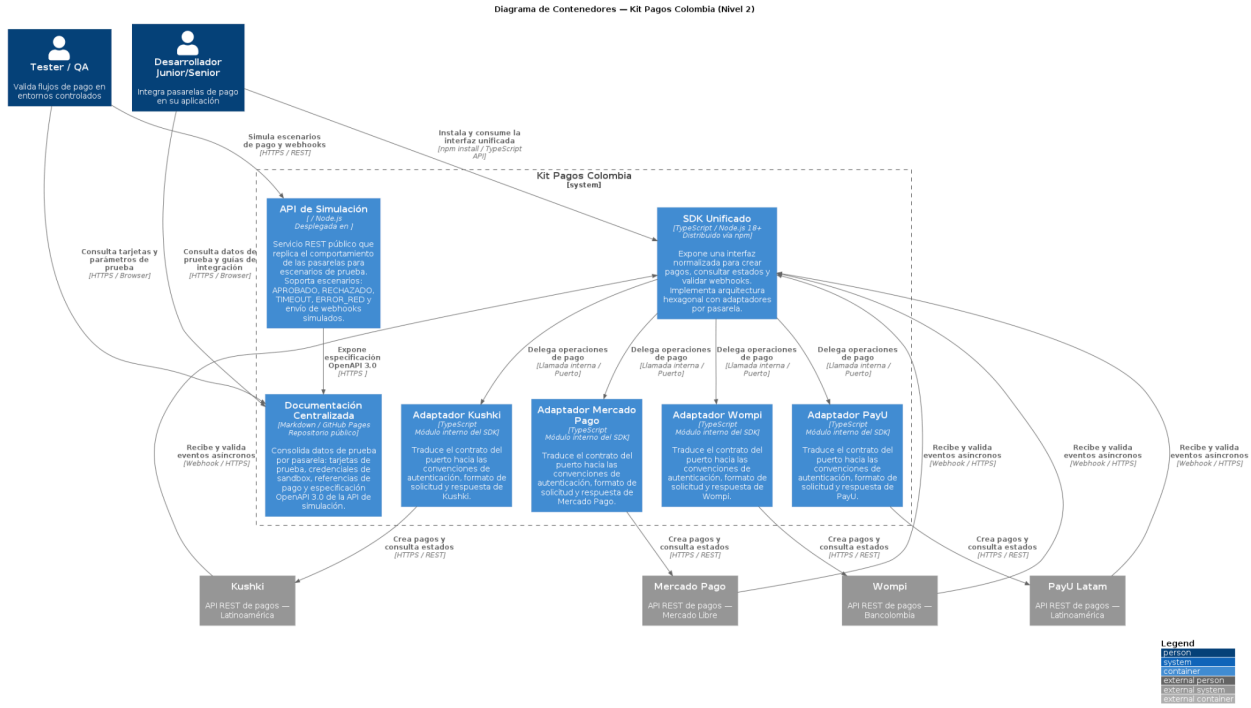


Figura 4. Diagrama de Contenedores de Kit Pagos Colombia

El diagrama de contenedores descompone el sistema Kit Pagos Colombia en sus unidades desplegables y ejecutables, mostrando cómo se distribuyen las responsabilidades entre ellas y cómo se comunican entre sí y con los sistemas externos.

8.1.1. Contenedores del sistema

El sistema está compuesto por tres contenedores principales que operan de forma complementaria.

El **SDK Kit Pagos Colombia** es el componente central del framework. Está desarrollado en TypeScript y se distribuye como un paquete npm compatible con Node.js 18 LTS en adelante. Su función principal es ofrecer una interfaz unificada para que los desarrolladores puedan crear pagos, consultar el estado de las transacciones y validar webhooks, sin tener que implementar de manera independiente la integración con cada pasarela.

Internamente, el SDK utiliza una arquitectura hexagonal y cuenta con cuatro adaptadores, uno para cada pasarela soportada: Wompi, Rapyd, Mercado Pago y Kushki. Cada adaptador se encarga de transformar el contrato definido por el SDK al formato y las reglas específicas de la pasarela correspondiente.

Para utilizarlo, el desarrollador instala el paquete mediante npm y configura las credenciales de las pasarelas que necesita. Cuando se trabaja en modo simulación, los adaptadores envían las solicitudes a la API de Simulación en lugar de comunicarse con las APIs reales de los proveedores.

Los cuatro adaptadores —Wompi, Rapyd, Mercado Pago y Kushki— forman parte del mismo paquete npm y están implementados como módulos internos del SDK. Por esta razón, no se consideran contenedores independientes: no tienen un proceso propio ni pueden desplegarse de manera aislada. La responsabilidad específica de cada uno se detalla posteriormente en el diagrama de componentes de Nivel 3.

La **API de Simulación** es un servicio REST desarrollado con Fastify y Node.js 18, desplegado en Render y disponible mediante una URL pública HTTPS. Su objetivo es simular el comportamiento de las cuatro pasarelas para los principales escenarios utilizados durante las pruebas, evitando la necesidad de utilizar credenciales o realizar transacciones reales.

El simulador permite configurar cinco escenarios mediante headers de control: **APROBADO, RECHAZADO, FONDOS_INSUFICIENTES, TIMEOUT y ERROR_RED**. También permite generar webhooks sintéticos firmados hacia la URL indicada por el comercio mediante una invocación manual. Estos webhooks buscan reproducir el formato y el algoritmo de firma utilizado por cada proveedor.

El Tester o equipo de QA interactúa directamente con este servicio mediante HTTPS para validar los diferentes flujos de pago dentro de un entorno controlado. La documentación de la API se encuentra disponible mediante OpenAPI 3.0 en el endpoint **/docs**, por lo que no es necesario disponer de un contenedor adicional para la documentación del servicio.

La **Documentación Centralizada** corresponde a un contenedor estático publicado mediante GitHub Pages y compuesto principalmente por archivos Markdown. Su propósito es reunir en un solo lugar la información necesaria para realizar pruebas con cada pasarela, incluyendo tarjetas de prueba, credenciales de sandbox, referencias de pago y tokens de acceso.

La documentación está organizada por proveedor y tipo de dato, permitiendo que tanto el Desarrollador como el Tester puedan consultar la información desde un navegador mediante HTTPS y encontrar rápidamente los datos necesarios para el desarrollo y las pruebas.

8.1.2. Relaciones entre contenedores

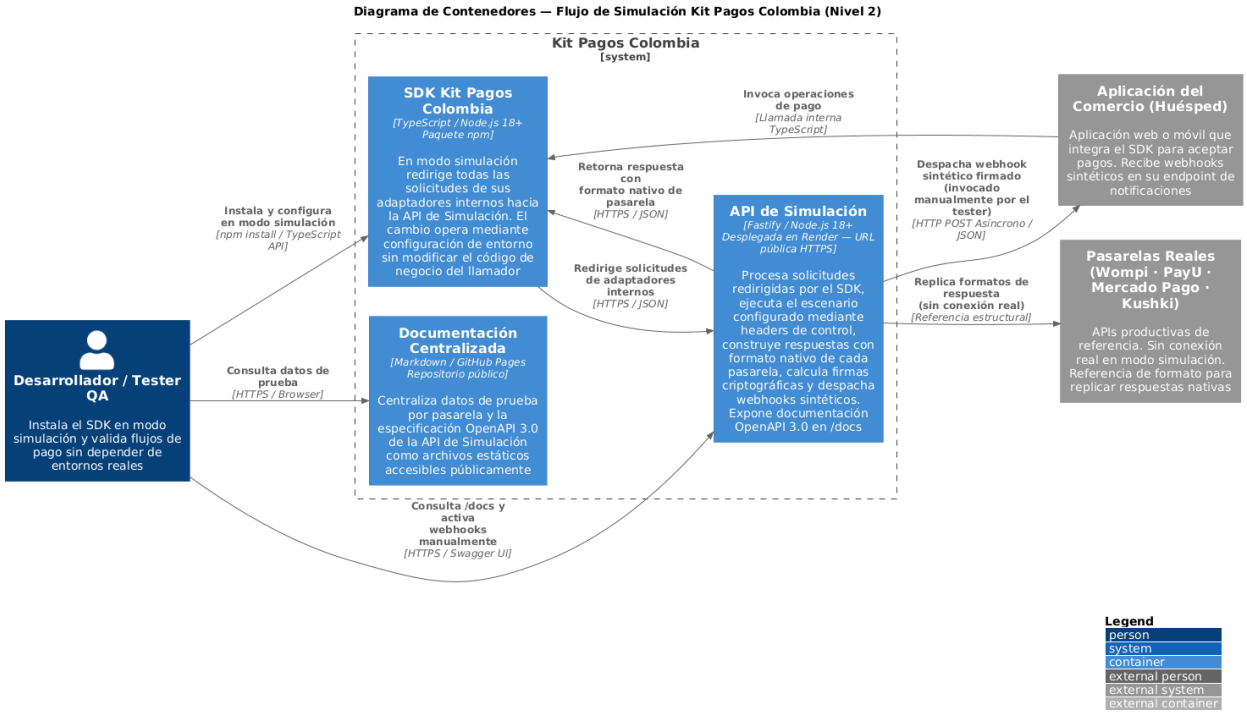
En producción, el SDK se comunica directamente con las pasarelas de pago mediante HTTPS. Para las pruebas, las solicitudes se redirigen hacia la API de Simulación, también mediante HTTPS y utilizando JSON. La API de Simulación incluye la especificación OpenAPI 3.0 en el endpoint **/docs**, desde donde se pueden consultar los endpoints y estructuras disponibles. En el flujo productivo, las pasarelas pueden enviar eventos de forma asíncrona hacia la aplicación mediante webhooks sobre HTTPS.

8.2 Diagrama de Contenedores — API de Simulación (Nivel 2)

Figura 5. Diagrama de Contenedores de la API de Simulación

Este diagrama complementa el anterior con un enfoque específico en el flujo de simulación, mostrando con mayor detalle cómo interactúan el SDK, la API de Simulación, la Documentación Interactiva y la Aplicación del Comercio durante las pruebas de desarrollo.

8.2.1. Contenedores en el contexto de simulación



El **SDK Kit Pagos Colombia** aparece en este diagrama como el cliente de la API de Simulación. Cuando se configura en modo simulación, sus adaptadores redirigen las solicitudes hacia el simulador en lugar de comunicarse con las APIs reales de los proveedores.

Este cambio se realiza mediante la configuración del entorno, por lo que la aplicación que utiliza el SDK no necesita modificar su código para pasar de un entorno de simulación a uno de producción.

La **API de Simulación Desplegada** es el componente principal de este flujo. Recibe las solicitudes enviadas por los adaptadores del SDK y determina la respuesta que debe generar según el escenario configurado mediante los headers de control.

A partir de esta información, la API genera una respuesta siguiendo el formato correspondiente a la pasarela que se está simulando. También permite generar webhooks sintéticos firmados hacia la Aplicación del Comercio cuando el Tester solicita explícitamente su envío.

La API expone además la especificación OpenAPI 3.0 mediante el endpoint **/docs**. Desde allí, tanto el Desarrollador como el Tester pueden consultar los endpoints disponibles, los headers aceptados, los escenarios de simulación y ejemplos de respuesta. Esta documentación puede consultarse directamente desde el navegador o utilizarse como referencia para realizar pruebas desde herramientas como Postman.

El servicio se encuentra desplegado en Render y cuenta con una URL pública mediante HTTPS, por lo que puede ser utilizado tanto por el SDK como directamente desde herramientas externas sin necesidad de realizar una configuración local del simulador.

La **Aplicación del Comercio (Huésped)** corresponde al sistema que integra el SDK para gestionar los pagos. Dentro del flujo de simulación, esta aplicación recibe los webhooks generados por la API de Simulación.

Para esto, la aplicación debe disponer de un endpoint de notificaciones al cual el simulador pueda enviar los eventos mediante HTTP POST. Los eventos incluyen la firma correspondiente para representar el comportamiento que tendría una pasarela real al enviar una notificación en producción.

8.2.2. Relaciones entre contenedores en el flujo de simulación

El Desarrollador o Tester instala el SDK mediante npm y lo configura para trabajar en modo simulación. La Aplicación del Comercio utiliza las operaciones de pago expuestas por el SDK y, dependiendo de la configuración, los adaptadores internos envían las solicitudes hacia la API de Simulación mediante HTTPS y JSON.

La API procesa cada solicitud de acuerdo con el escenario configurado y genera la respuesta correspondiente. Esta respuesta es enviada nuevamente al SDK, donde se normaliza para que la Aplicación del Comercio pueda trabajar con una entidad **Transaction** común, independientemente de la pasarela que se esté simulando.

Cuando se requiere probar el flujo de notificaciones, el Tester puede solicitar manualmente el envío de un webhook mediante el endpoint **/v1/sim/webhooks/trigger**. La API de Simulación genera entonces el evento correspondiente y lo envía mediante HTTP POST al endpoint configurado en la Aplicación del Comercio.

Por otra parte, el Desarrollador o Tester puede consultar la documentación del simulador desde **/docs**. Esto permite revisar y probar los endpoints disponibles sin tener que consultar directamente el código fuente de la API.

9. Vista de componentes

9.1 Diagrama de Componentes — SDK Kit Pagos Colombia (Nivel 3)

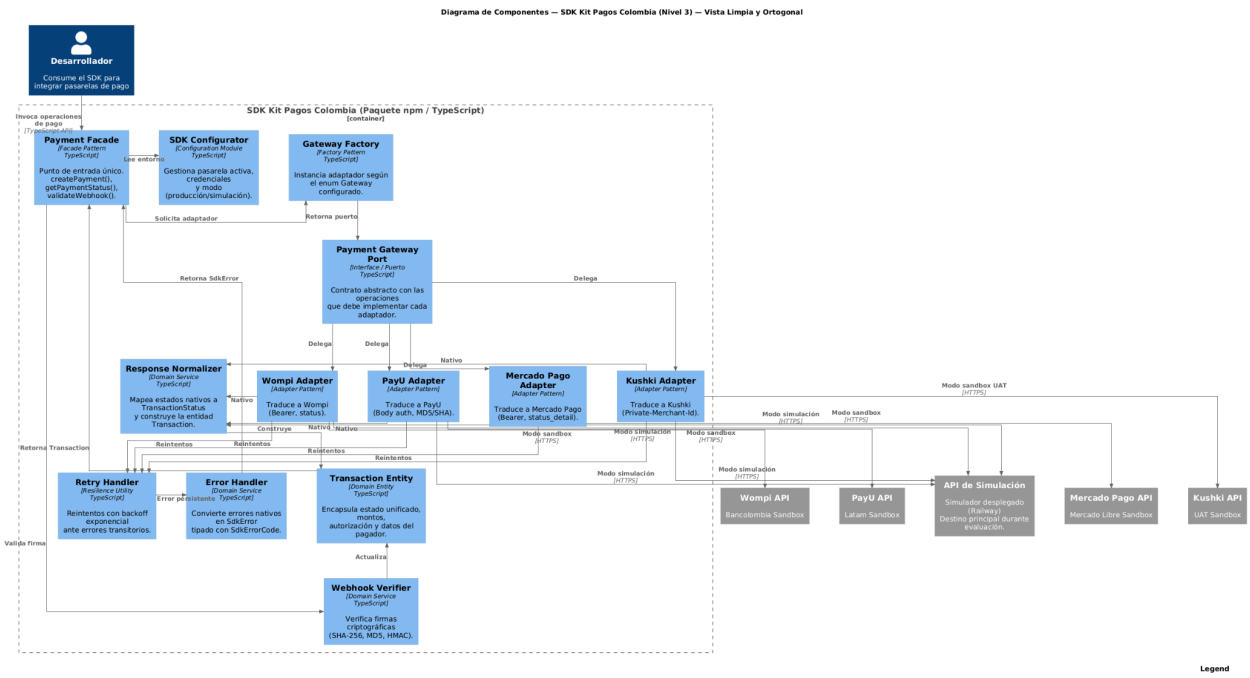


Figura 6. Diagrama de Componentes del SDK

El SDK es el contenedor de mayor complejidad arquitectónica del sistema. Su diseño interno sigue la arquitectura hexagonal (Ports & Adapters) que organiza el sistema alrededor de un núcleo de dominio independiente, comunicado con el exterior únicamente a través de interfaces abstractas. El diagrama expone once componentes con responsabilidades claramente delimitadas, cuyo alcance de implementación fue definido en función del tiempo disponible y de la prioridad de cada pasarela dentro del proyecto.

9.1.1 Payment Facade

Es el punto de entrada único del SDK y el único componente con el que el desarrollador interactúa directamente. Expone tres métodos públicos: **createPayment(), getPaymentStatus() y validateWebhook()**. Su propósito es ocultar la complejidad interna del sistema detrás de una interfaz simple y predecible, implementando el patrón Facade del catálogo GoF. Retorna entidades Transaction normalizadas al desarrollador o excepciones SdkError tipadas en caso de fallo.

9.1.2. SDK Configurator

Gestiona toda la configuración del SDK en un único punto: pasarela activa, credenciales por proveedor, modo de entorno y parámetros del mecanismo de reintento. El switch entre modo producción y modo simulación vive en este componente. Cuando el modo es simulación, los adaptadores redirigen sus solicitudes hacia la API de Simulación, que constituye el entorno principal de validación durante la fase de evaluación del proyecto.

9.1.3. Gateway Factory

Recibe el enum **Gateway,** con valores WOMPI, RAPYD, MERCADOPAGO o KUSHKI y retorna la instancia del adaptador correspondiente, implementando el patrón Factory del catálogo GoF. Cambiar de pasarela es una operación de configuración: el desarrollador modifica el parámetro de inicialización del SDK y el Factory instancia el adaptador correcto sin que ningún otro componente deba cambiar.

9.1.4. Payment Gateway Port

Es la interfaz abstracta que define el contrato que todo adaptador debe cumplir, especificando las tres operaciones disponibles sin conocimiento de ningún proveedor específico. Es el elemento central de la arquitectura hexagonal y el que hace posible la extensibilidad hacia nuevos proveedores sin modificar el núcleo del sistema.

9.1.5. Wompi Adapter, Rapyd Adapter, Mercado Pago Adapter y Kushki Adapter

Cada uno implementa el Payment Gateway Port y traduce su contrato hacia las convenciones del proveedor que representa. Dentro del alcance del proyecto, los cuatro adaptadores se encuentran implementados y validados en los respectivos ambientes sandbox, con distintos niveles de prioridad. El Wompi Adapter es el de mayor prioridad y profundidad de validación, dado que Wompi es la pasarela con mayor penetración en el mercado colombiano, la mejor documentación técnica y el sandbox de más fácil acceso. Los adaptadores de Rapyd, Mercado Pago y Kushki tienen el mismo nivel de completitud funcional, aunque su validación en sandbox fue progresiva durante el período de desarrollo. En modo simulación, los cuatro adaptadores redirigen sus solicitudes hacia la API de Simulación mediante HTTPS incluyendo los headers de control de escenario.

9.1.6. Response Normalizer

Recibe la respuesta nativa de cualquier adaptador y la transforma al modelo de dominio unificado del SDK, mapeando los estados crudos de cada proveedor al enum **TransactionStatus** con valores APPROVED, DECLINED, PENDING, EXPIRED, VOIDED y ERROR, y construyendo la entidad Transaction correspondiente. Este componente es el que hace medible el impacto del framework mediante las métricas CK: al centralizar la normalización, el código cliente no necesita referenciar los tipos de respuesta de ninguna pasarela, lo que reduce directamente su CBO.

9.1.7. Webhook Verifier

Verifica la autenticidad de los webhooks entrantes de cada pasarela mediante el método **verify(payload, headers, secret, gateway): boolean**. Dentro del alcance del proyecto, la implementación completa del algoritmo de verificación corresponde a Wompi mediante SHA-256, que es la pasarela de referencia. Para Rapyd y Kushki se implementa la verificación con el algoritmo correspondiente HMAC-SHA256, en un nivel de profundidad suficiente para validar los webhooks generados por el simulador durante la fase de evaluación. Por otro lado, para Mercado Pago el algoritmo es el mismo pero el resultado viaja en un encabezado con un formato propio (ts=...,v1=...). El detalle exacto de cada fórmula está documentado en el Lenguaje Ubicuo del proyecto.

9.1.8. Transaction Entity

Es la entidad central del dominio del SDK. Encapsula el estado unificado de una transacción (**rawStatus, authorizationCode, payer, amount, currency**) e implementa los métodos de consulta **isApproved()**, **isPending()** e **isFinal()** que permiten al desarrollador tomar decisiones de negocio sin depender de los estados crudos de ninguna pasarela.

9.1.9. Retry Handler

Implementa backoff exponencial con valores configurables (por defecto tres reintentos con intervalos de 1, 2 y 4 segundos) ante errores transitorios como timeout de red o respuestas HTTP 5xx. Distingue entre errores transitorios candidatos a reintento y errores definitivos que no se recuperarán con intentos adicionales. Si se supera el máximo de intentos, pasa el error al Error Handler con código **MAX_RETRIES_EXCEEDED**.

9.1.10. Error Handler

Convierte cualquier error en una excepción **SdkError** tipada con código normalizado del enum **SdkErrorCode,** que incluye INVALID_CREDENTIALS, GATEWAY_TIMEOUT, CONNECTION_FAILED, WEBHOOK_SIGNATURE_INVALID y MAX_RETRIES_EXCEEDED entre otros, mensaje descriptivo en español e inglés y pasarela de origen. Garantiza que el desarrollador nunca reciba excepciones no manejadas ni errores en formato nativo de ninguna pasarela.

9.2 Diagrama de Componentes — API de Simulación (Nivel 3)

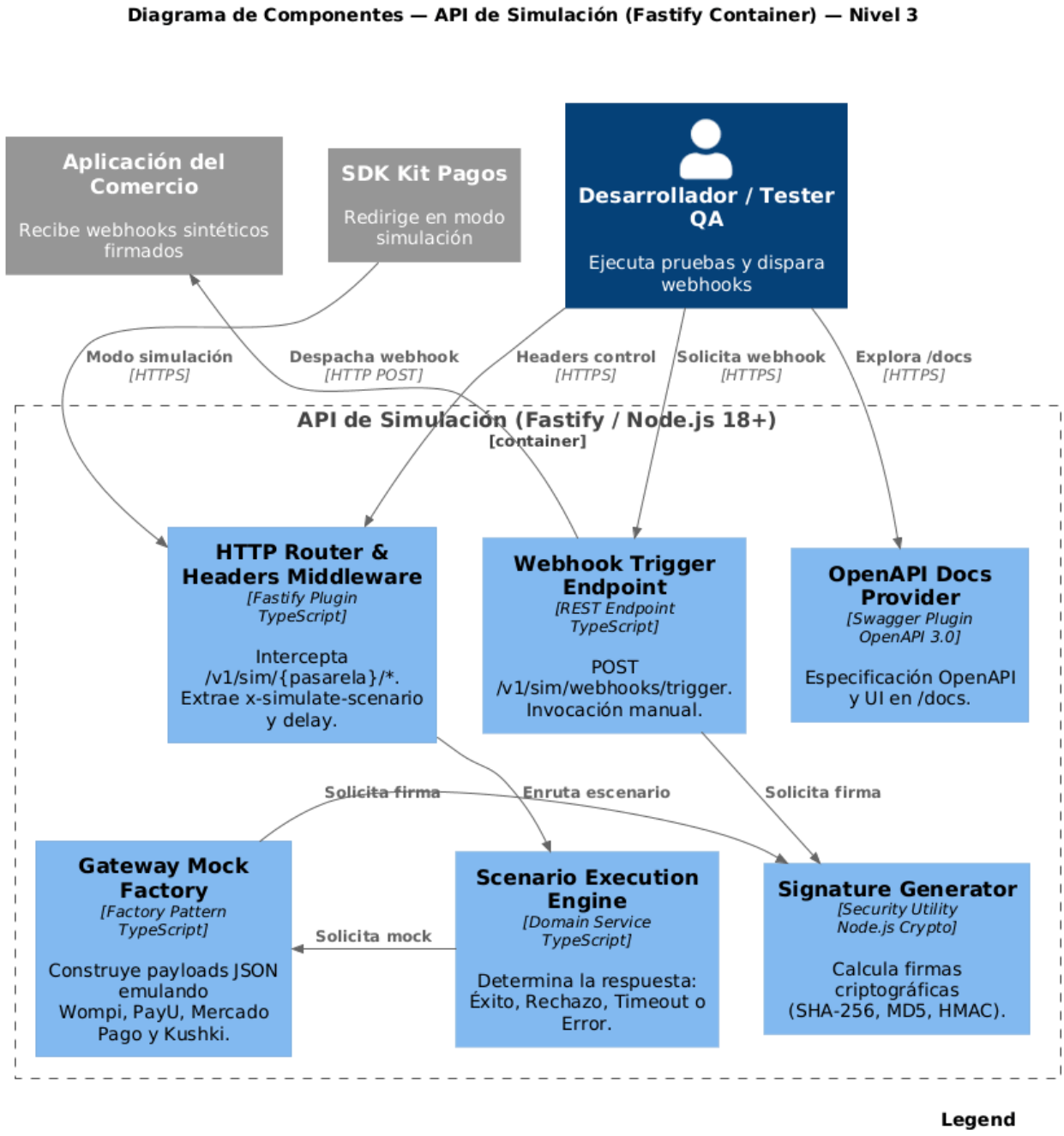


Figura 7. Diagrama de Componentes de la API de Simulación

La API de Simulación es un servicio Fastify sobre Node.js 18 cuya arquitectura interna sigue un flujo de procesamiento secuencial con responsabilidades claramente delimitadas entre cinco componentes. El diagrama muestra ese flujo de izquierda a derecha, desde la recepción de la solicitud hasta la generación

de la respuesta. El alcance del simulador fue definido deliberadamente para maximizar la utilidad durante la fase de evaluación del proyecto dentro del tiempo disponible.

9.2.1. HTTP Router & Headers Middleware

Es la puerta de entrada del simulador, implementado como un plugin Fastify. Intercepta todas las solicitudes REST entrantes e identifica la pasarela de destino a partir de la estructura de la URL: **/v1/sim/wompi/*, /v1/sim/rapyd/*, /v1/sim/mercadopago/*** o **/v1/sim/kushki/*. Extrae los dos headers de control que gobiernan el comportamiento del simulador: x-simulate-scenario**, que indica el escenario a ejecutar, y **x-simulate-delay**, que especifica los milisegundos de latencia a introducir. Las solicitudes pueden provenir del SDK Kit Pagos Colombia en modo simulación o directamente del Desarrollador o Tester mediante herramientas REST.

9.2.2. Scenario Execution Engine

Es el componente de decisión del simulador. Para los escenarios de negocio (APROBADO, RECHAZADO y FONDOS_INSUFICIENTES) delega la construcción del payload al Gateway Mock Factory. Para TIMEOUT introduce la latencia especificada antes de responder, permitiendo verificar que el Retry Handler del SDK maneja correctamente los tiempos de espera prolongados. Para ERROR_RED retorna directamente una respuesta HTTP 500 o 502 sin invocar al Mock Factory, simulando la caída del servidor del proveedor.

9.2.3. Gateway Mock Factory

Construye payloads JSON que replican con exactitud la estructura de las respuestas nativas de cada pasarela para los escenarios de éxito y rechazo. Está compuesto por cuatro fábricas internas (WompiMockFactory, RapydMockFactory, MercadoPagoMockFactory y KushkiMockFactory) cada una con conocimiento del vocabulario de campos y valores propios de su proveedor. La precisión de este componente es crítica para la validación del framework: si el formato del Mock no coincide con el de la pasarela real, el Response Normalizer del SDK fallará en los escenarios de prueba.

9.2.4. Signature Generator

Calcula la firma criptográfica que acompaña a los webhooks simulados, de modo que el Webhook Verifier del SDK pueda verificarla. Dentro del alcance del proyecto, la implementación completa corresponde a Wompi mediante SHA-256. Para Rapyd, Mercado Pago y Kushki se implementa la firma con sus algoritmos respectivos en un nivel suficiente para que los webhooks generados por el simulador pasen la verificación del SDK durante la fase de evaluación. En el caso de Rapyd, ese algoritmo es HMAC-SHA256 con el resultado codificado en Base64, distinto del MD5 que usaba la pasarela que este proyecto documentaba originalmente en este mismo lugar. Este componente no opera de forma automática ni asíncrona, es invocado por el Webhook Trigger Endpoint cuando el desarrollador solicita explícitamente el envío de un webhook sintético.

9.2.5. Webhook Trigger Endpoint

Es el componente que gestiona el envío de webhooks sintéticos de forma controlada y explícita. A diferencia de un despachador automático, este endpoint requiere que el desarrollador o tester lo invoque manualmente mediante una solicitud POST a **/v1/sim/webhooks/trigger**, indicando la URL destino, el tipo de evento y la pasarela a simular. Esta decisión de diseño responde a una restricción práctica del contexto de evaluación académica: los proyectos prototípicos que integran el framework corren típicamente en entornos locales sin URL pública fija, lo que hace inviable el dispatch automático sin una solución de tunelización adicional. Al requerir invocación manual, el

componente elimina esa dependencia sin sacrificar la capacidad de probar el flujo completo de validación de webhooks.

9.2.6. OpenAPI Documentation Provider

Sirve automáticamente la especificación OpenAPI 3.0 del simulador en el endpoint **/docs** mediante el plugin Swagger de Fastify. No requiere mantenimiento manual, cada modificación en los endpoints del simulador se refleja automáticamente en la especificación. Constituye uno de los entregables formales del proyecto definidos en la propuesta, lo que hace que este componente produzca directamente un artefacto evaluable sin trabajo adicional.

10. Vista de procesos

Esta sección describe el comportamiento dinámico de los flujos más críticos del sistema en relación con sus atributos de calidad, complementando las vistas estáticas de componentes (secciones 8 y 9) con Diagramas de Secuencia (C4 Dynamic Diagram) que muestran cómo colaboran esos componentes en tiempo de ejecución. Se agrupan en dos conjuntos: los que describen el comportamiento interno del SDK frente a la aplicación del comercio, y los que describen el comportamiento de la API de Simulación frente a cada escenario de prueba configurable.

10.1 Diagramas de Secuencia — SDK

10.1.1 Creación de Pago (pendiente + webhook)

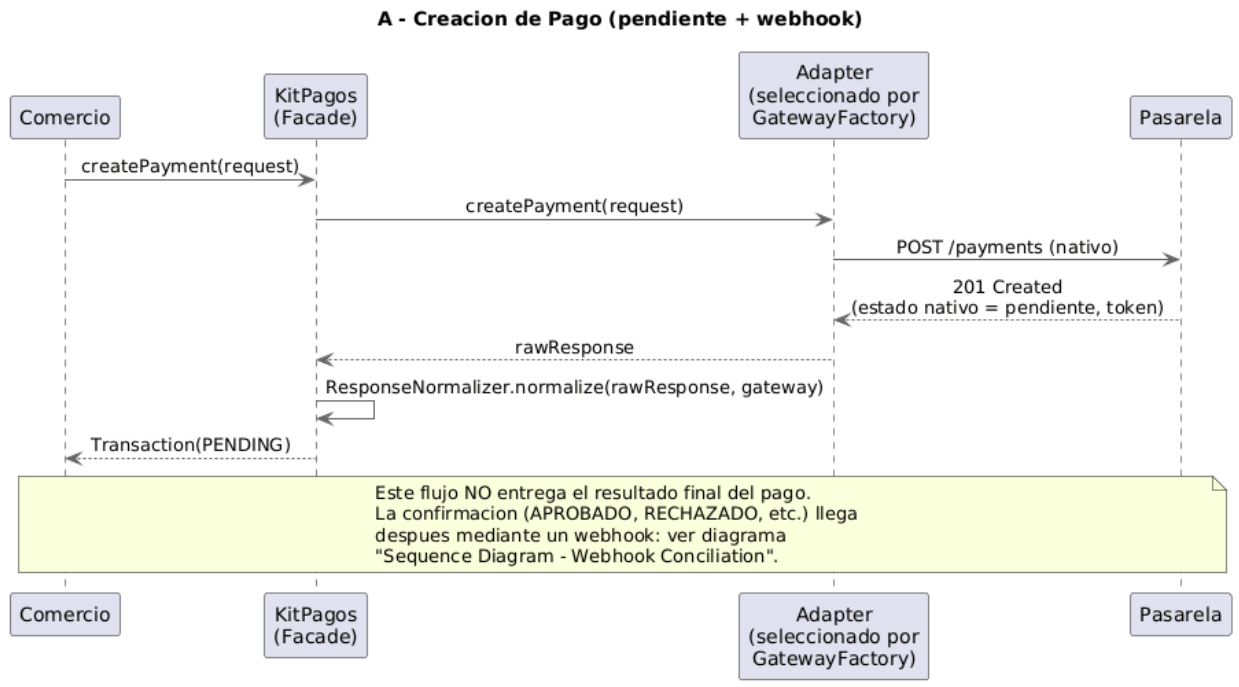


Figura 8. Diagrama de Secuencia del SDK de creación de pago

Este diagrama ilustra el flujo estándar de creación de un pago cuando la pasarela responde de forma asíncrona. El Comercio invoca el método createPayment(request) sobre la fachada principal KitPagos (Facade), la cual delega la llamada enviando createPayment(request) hacia el Adapter (previamente seleccionado mediante el

GatewayFactory). El Adapter realiza una petición nativa POST /payments (nativo) hacia la API de la pasarela (o de simulación) y recibe una respuesta 201 Created que incluye el token y el estado nativo equivalente a pendiente.

Posteriormente, el Adapter devuelve rawResponse a KitPagos (Facade), componente que invoca internamente a ResponseNormalizer.normalize(rawResponse, gateway) para transformar la respuesta cruda en la entidad del dominio Transaction con estado PENDING. Finalmente, la fachada retorna la instancia Transaction(PENDING) de vuelta al Comercio.

El diagrama deja explícito, mediante una nota, que este flujo no entrega el resultado final del pago; la confirmación definitiva (APROBADO, RECHAZADO, etc.) llega posteriormente de manera asíncrona mediante un webhook, cuyo procesamiento se documenta por separado en el diagrama de conciliación por webhook. Este comportamiento es relevante para el atributo de calidad de confiabilidad, ya que evidencia que el SDK no bloquea al comercio esperando una resolución síncrona que la pasarela no garantiza en su llamada inicial..

10.1.2 Pago Síncrono (Kushki — Tarjeta)

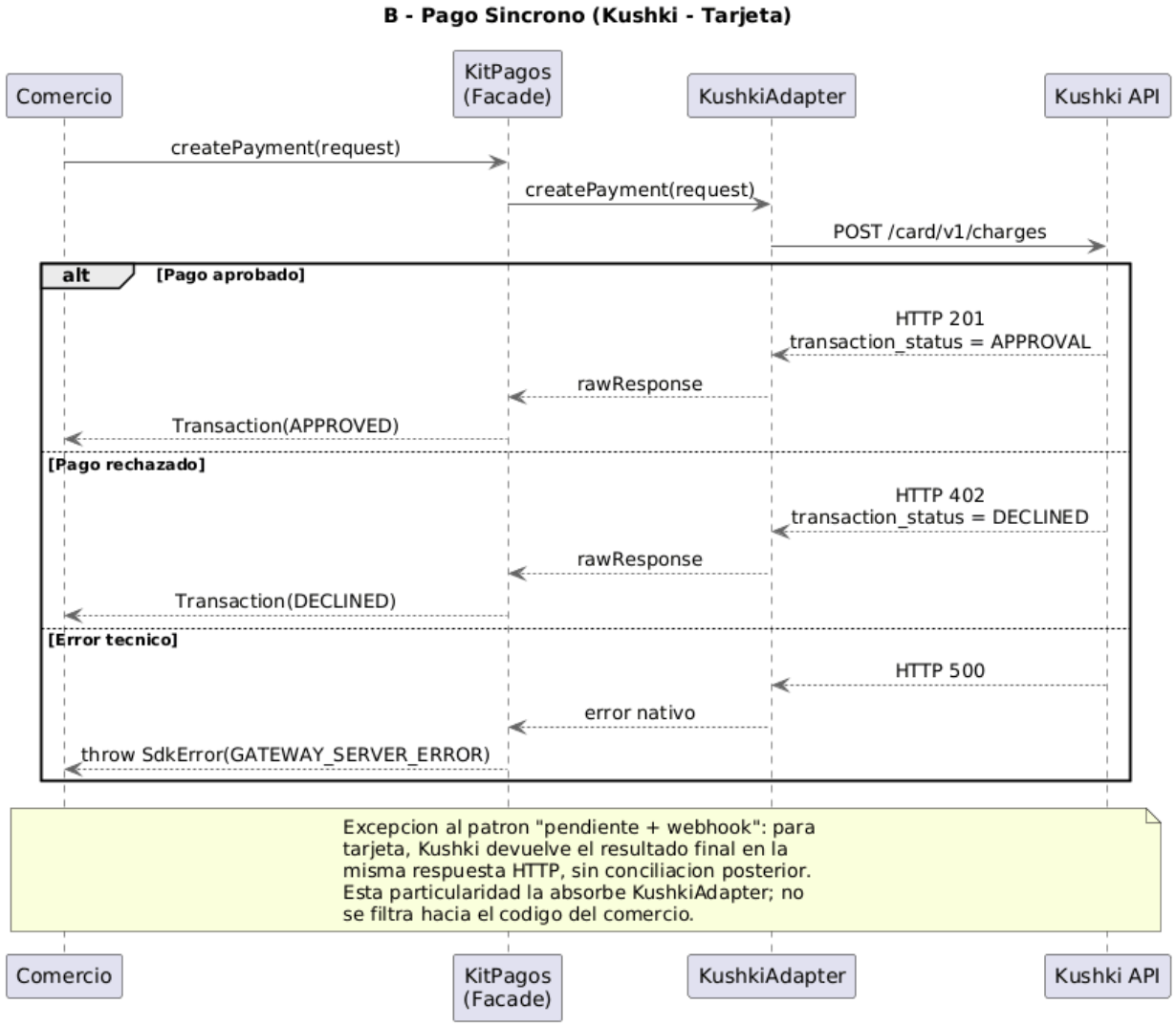


Figura 9. Diagrama de Secuencia del SDK de creación de pago síncrona

Este diagrama documenta la excepción al flujo anterior: para operaciones con tarjeta sobre Kushki, la pasarela sí devuelve el resultado final de la transacción en la misma respuesta HTTP, sin requerir conciliación posterior por webhook. El diagrama usa un bloque alt para mostrar las tres ramas posibles ante la llamada POST /charges del KushkiAdapter: pago aprobado (HTTP 201, PaymentResult(APPROVED)), pago rechazado (HTTP 402, PaymentResult(REJECTED)) y error técnico (HTTP 500, traducido a SdkError(GATEWAY_ERROR)). Este diagrama es importante porque advierte a quien implemente o integre el SDK que el patrón de "pendiente + webhook" documentado en 10.1.1 **no es universal**: el comportamiento síncrono de Kushki para tarjetas es una particularidad del proveedor que el KushkiAdapter debe manejar de forma distinta a los demás, sin que esa diferencia se filtre hacia el código del comercio gracias a la normalización de PaymentResult.

10.1.3 Conciliación por Webhook (asíncrono)

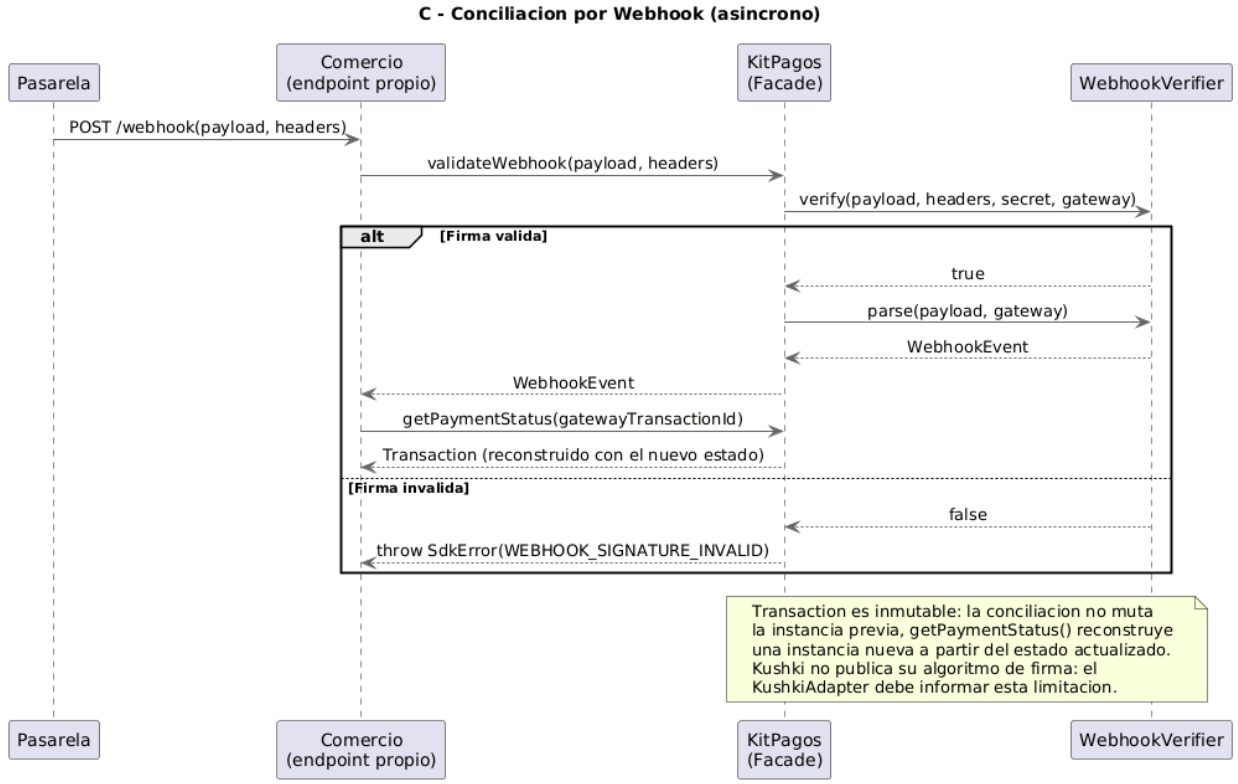


Figura 10. Diagrama de Secuencia del SDK de conciliación por webhook

Este diagrama complementa al de Creación de Pago (10.1.1) mostrando cómo se resuelve finalmente una transacción que quedó en estado PENDING. La Pasarela envía un POST /webhook con el payload y los headers de firma hacia el WebhookController, que delega en ProcesarWebhookUseCase. Este último invoca verifySignature() sobre el Adapter correspondiente antes de procesar cualquier dato del evento, siguiendo el bloque alt que separa el camino de **firma válida** (se parsea el webhook, se actualiza la transacción y se retorna HTTP 200) del camino de **firma inválida** (se descarta el evento y se retorna HTTP 401 con SdkError(WEBHOOK_SIGNATURE_INVALID), sin llegar a actualizar ninguna transacción). Este diagrama materializa directamente el ADR-04 (verificación criptográfica de firmas) y es crítico para el atributo de calidad de **seguridad**: deja explícito que ningún dato de un webhook se procesa antes de validar su origen. La nota final, sobre Kushki no publicar su algoritmo de firma,

documenta una limitación conocida que el Adapter debe comunicar explícitamente al comercio, en línea con el riesgo técnico ya identificado sobre la implementación parcial de firmas.

10.2 Diagramas de Secuencia — API de Simulación

10.2.1 Pago Exitoso (Escenario 1)

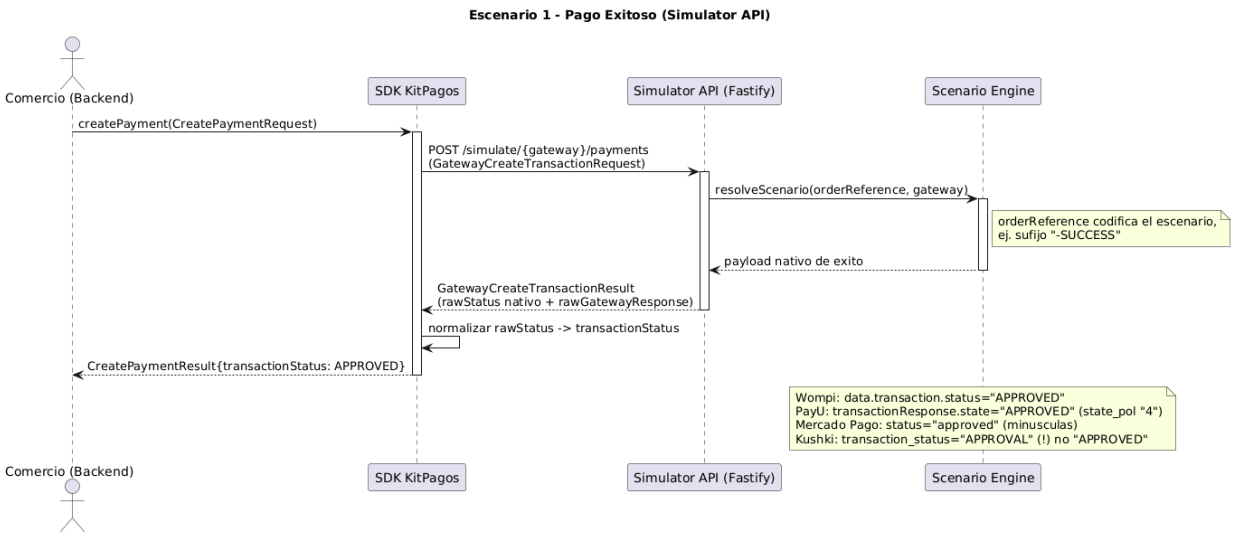


Figura 11. Diagrama de Secuencia de la API de Simulación de pago exitoso

Muestra el flujo base de la API de Simulación: el Comercio invoca createPayment() sobre el SDK, que traduce la llamada en un POST /simulate/{gateway}/payments hacia la API. El Scenario Engine resuelve el escenario a partir del orderReference recibido (una nota aclara que el propio orderReference codifica el escenario esperado, por ejemplo con el sufijo "-SUCCESS") y retorna el payload nativo de éxito. Este diagrama es especialmente relevante porque su nota final expone, con datos concretos, el problema central que resuelve el ADR-03: cada pasarela representa un pago aprobado con un vocabulario distinto (APPROVED en Wompi, CLO en Rapyd (acompañado del campo paid: true), approved en minúsculas en Mercado Pago, y el caso particular de Kushki con APPROVAL en lugar de APPROVED), lo que justifica la existencia del Response Normalizer.

10.2.2 Pago Rechazado (Escenario 2)

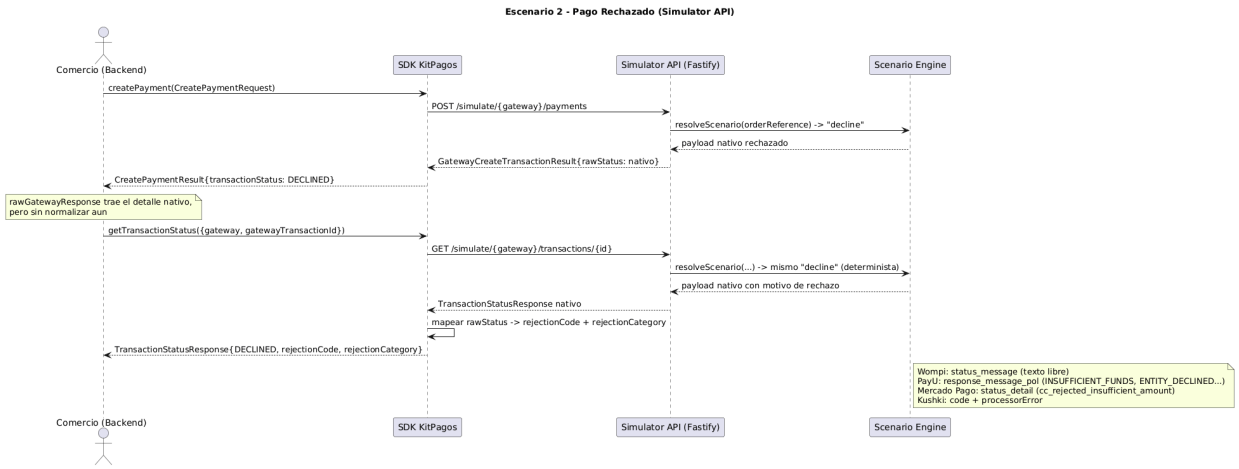


Figura 12. Diagrama de Secuencia de la API de Simulación de pago rechazado

Extiende el flujo anterior mostrando dos operaciones en secuencia: la creación del pago (que retorna un estado DECLINED normalizado) y la consulta posterior de estado mediante getTransactionStatus(), que el diagrama marca explícitamente como determinista (el Scenario Engine resuelve el mismo resultado de rechazo ante la misma referencia). Es el diagrama donde se evidencia el mapeo hacia rejectionCode y rejectionCategory descrito en el modelo de dominio, y su nota final documenta, igual que en el escenario anterior, la heterogeneidad real del motivo de rechazo entre proveedores (texto libre en Wompi, el código compuesto ERROR_PROCESSING_CARD seguido del código de red entre corchetes en Rapyd (por ejemplo, ERROR_PROCESSING_CARD - [51] para fondos insuficientes), status_detail en Mercado Pago, y la combinación de code + processorError en Kushki), justificando por qué esa normalización debe vivir en el SDK y no quedar como responsabilidad del comercio.

10.2.3 Error de Red (Escenario 3)

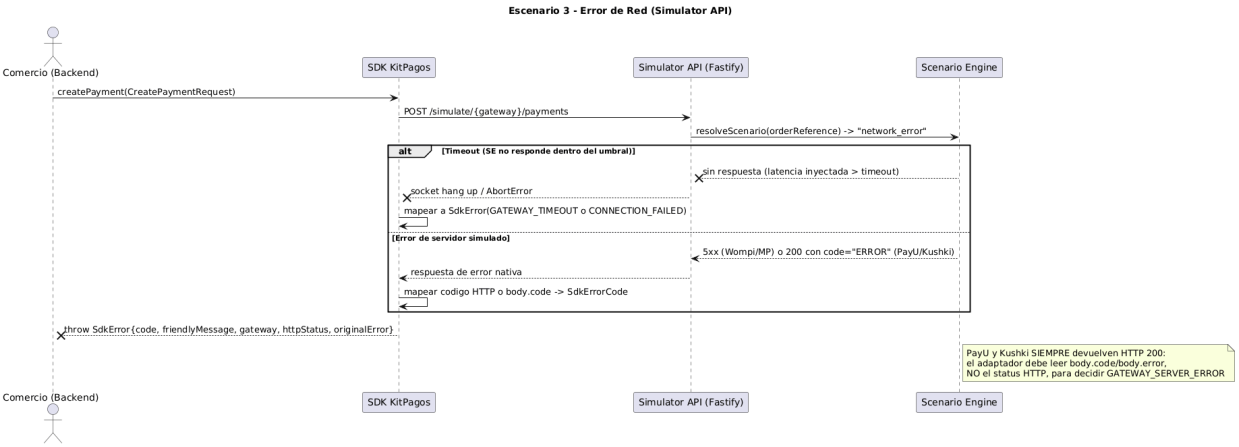


Figura 13. Diagrama de Secuencia de la API de Simulación de error de red

Documenta el escenario más complejo de los cuatro, mediante un bloque alt con dos ramas. La primera es **timeout**: la API de Simulación deliberadamente no responde dentro del umbral configurado, y el SDK debe capturar el corte de conexión y mapearlo a SdkError(GATEWAY_TIMEOUT) o CONNECTION_FAILED). La segunda es **error de servidor simulado**, donde la nota final señala una particularidad crítica para la implementación de los Adapters: Kushki siempre devuelve HTTP 200 incluso ante un error, por lo que su Adapter no puede confiar en el código de

estado HTTP para detectar la falla y debe inspeccionar explícitamente el campo body.code o body.error del payload. Rapyd es distinto en este punto: sí devuelve códigos HTTP reales (4xx o 5xx) cuando la operación falla, pero el detalle específico del error igual viaja dentro del cuerpo de la respuesta, en un objeto status anidado (status.error_code, status.message), así que su Adapter también necesita leer el body completo, solo que puede apoyarse primero en el código HTTP como una señal confiable, algo que el Adapter de Kushki no puede hacer. Este diagrama es directamente relevante para el atributo de calidad de **confiabilidad** y para el ADR-05 (reintentos), ya que ambas ramas terminan siendo clasificadas por el Retry Handler del SDK como errores transitorios candidatos a reintento.

10.2.4 Notificación Webhook (Escenario 4)

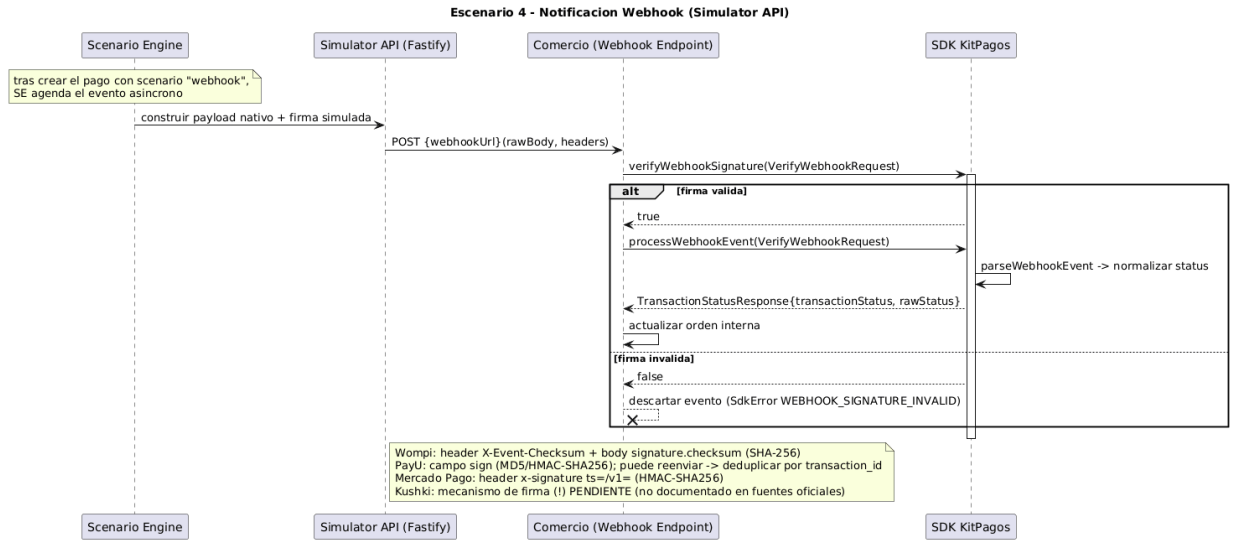


Figura 14. Diagrama de Secuencia de la API de Simulación de Notificación webhook

Muestra el lado de la API de Simulación que dispara webhooks sintéticos: tras crear un pago con escenario "webhook", el Scenario Engine construye el payload nativo con firma simulada y lo envía mediante POST al endpoint del Comercio. El diagrama usa un alt para mostrar la verificación de firma en el lado del SDK receptor (verifyWebhookSignature), diferenciando el camino de firma válida (que procesa el evento, normaliza el estado y permite al comercio actualizar su orden interna) del de firma inválida (que descarta el evento con SdkError(WEBHOOK_SIGNATURE_INVALID)). Es el complemento natural del diagrama de Conciliación por Webhook (10.1.3): mientras aquel muestra cómo el SDK procesa un webhook real, este muestra cómo la API de Simulación lo genera de forma controlada para poder ejercitar ese flujo durante las pruebas. La nota final resume el estado de implementación de la firma por pasarela (Wompi completo mediante SHA-256; Rapyd, Mercado Pago y Kushki implementados a un nivel suficiente para las pruebas del simulador, sin la robustez de un algoritmo validado contra la documentación oficial completa de cada proveedor"), documentando de forma trazable el riesgo técnico ya identificado sobre la verificación incompleta de firmas.

11. Vista física

De los tres componentes que conforman Kit Pagos Colombia (el SDK, la API de Simulación y la documentación centralizada de datos de prueba), únicamente la API de Simulación requiere una vista física propia dentro de este documento. El SDK se distribuye como un paquete de npm que se instala e importa directamente dentro del proceso de la aplicación del comercio que lo consume, por lo que no posee un ciclo de despliegue independiente ni una

infraestructura propia que documentar a este nivel. De manera similar, la documentación de datos de prueba corresponde a contenido estático alojado dentro del repositorio del proyecto, sin un componente en ejecución asociado. La API de Simulación, en cambio, sí constituye un servicio que debe permanecer desplegado y accesible públicamente durante la fase de evaluación del proyecto, ya que reproduce el comportamiento de las pasarelas reales para que tanto el equipo desarrollador como los evaluadores puedan ejecutar los escenarios de prueba sin depender de las plataformas de pago externas.

La Figura X presenta el Diagrama de Despliegue construido bajo la notación C4 para la API de Simulación. Este diagrama complementa al Diagrama de Contenedores presentado en la sección anterior, trasladando la anotación de infraestructura que allí aparecía únicamente como texto descriptivo hacia una representación explícita de nodos físicos y sus relaciones.

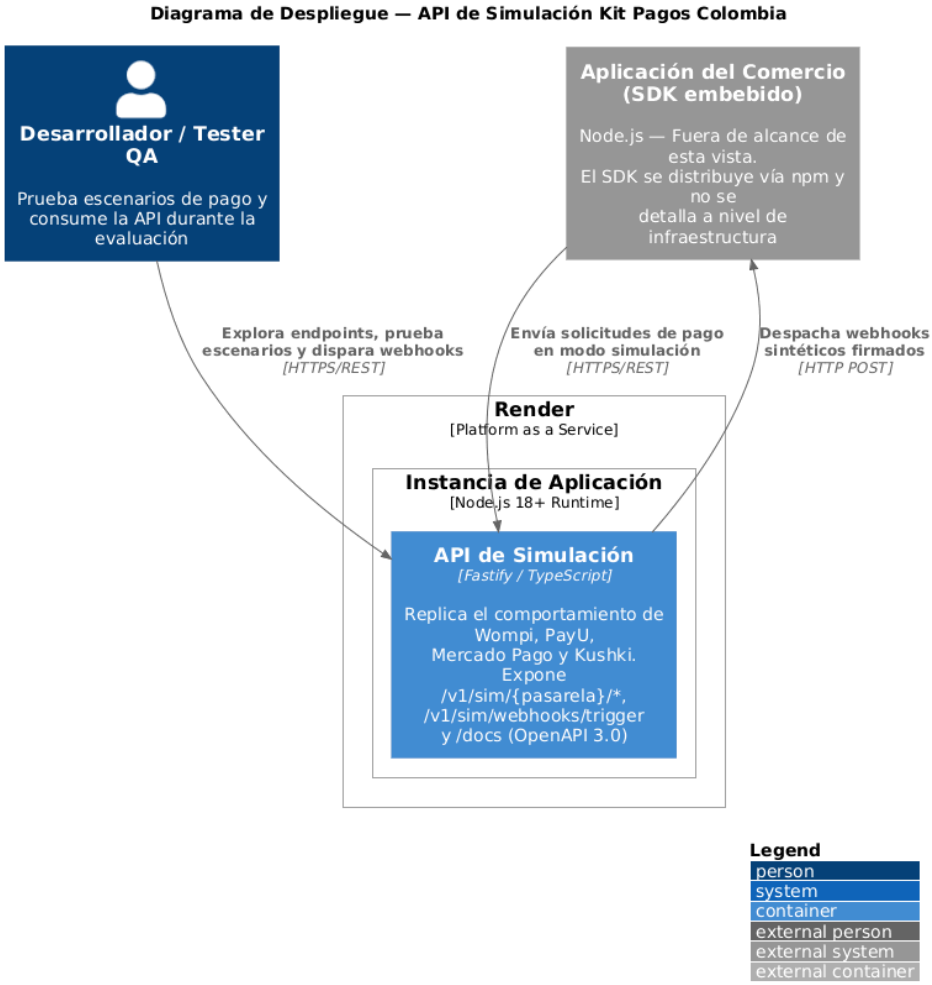


Figura 15. Diagrama de despliegue de la API de simulación

11.1. Actores externos

11.1.1. Desarrollador / Tester QA

Este actor es quien durante la fase de evaluación accede directamente a la API mediante un navegador o herramientas como Postman, con el fin de explorar los endpoints documentados, ejecutar los distintos escenarios de

prueba disponibles (aprobado, rechazado, fondos insuficientes, timeout y error de red) y disparar manualmente el envío de webhooks sintéticos.

11.1.2. Aplicación del comercio

Representada como un sistema externo debido a que, como se mencionó, el SDK embebido dentro de ella no forma parte del alcance de esta vista; su única relevancia aquí es que constituye el consumidor real de la API cuando el SDK opera en modo simulación, y a la vez es el receptor de los webhooks que la API despacha de forma asíncrona.

11.2. Infraestructura de despliegue

La API se aloja dentro de una plataforma en la nube de tipo PaaS, la alternativa seleccionada corresponde a Render. Dentro de dicha plataforma, la aplicación corre sobre una instancia con runtime de Node.js en su versión 18 o superior, que es el mismo requisito de compatibilidad definido para el resto del proyecto. Finalmente, dentro de ese runtime se ejecuta el contenedor lógico de la API de Simulación, construido sobre Fastify y TypeScript, que expone tanto los endpoints de simulación bajo el patrón */v1/sim/{pasarela}/** como el endpoint de disparo manual de webhooks */v1/sim/webhooks/trigger* y la documentación interactiva generada automáticamente en /docs mediante la especificación OpenAPI 3.0.

11.3. Relaciones y Flujos de Comunicación

Las relaciones del diagrama reflejan tres flujos de comunicación, todos ellos sobre HTTPS. El primero corresponde a la exploración y prueba directa que realiza el Desarrollador o Tester QA sobre la API desplegada. El segundo corresponde a las solicitudes de pago que la Aplicación del Comercio envía hacia la API cuando el SDK se encuentra configurado en modo simulación, replicando así el mismo contrato que tendría una solicitud dirigida a una pasarela real. El tercero es el envío asíncrono de webhooks sintéticos firmados desde la API hacia la Aplicación del Comercio, que permite validar que el mecanismo de verificación de firmas del SDK funcione correctamente frente a eventos generados por el simulador.

12. Modelo de datos

Kit Pagos Colombia no tiene un modelo de datos persistente que documentar en esta sección. No hay que confundirlo con el modelo de dominio de la sección 3: ese modelo describe estructuras que viven en memoria durante una sola llamada al SDK (Transaction, Amount, Currency, etc.). Esta sección, en cambio, habla de datos que el sistema guardaría de forma duradera entre solicitudes, y ese tipo de almacenamiento no existe en ninguno de los tres componentes del proyecto.

El SDK Unificado no guarda estado entre invocaciones. Como se explica en la vista física (sección 11), el SDK se distribuye como paquete de npm y se instala e importa directamente en la aplicación del comercio que lo usa. Por eso no tiene ciclo de despliegue propio ni infraestructura que documentar. Sus once componentes (sección 9.1) trabajan solo con los datos que reciben en cada llamada: el Payment Facade devuelve una entidad Transaction que el Response Normalizer construye en memoria a partir de la respuesta de la pasarela. Ningún componente escribe ni consulta un repositorio de datos.

La API de Simulación tampoco necesita persistencia. Sus seis componentes (sección 9.2) resuelven cada petición de forma determinista usando la información que viene en la propia solicitud, en concreto los headers x-simulate-scenario y x-simulate-delay. No guarda historial entre solicitudes ni mantiene estado compartido.

13. Registros de Decisiones Arquitectónicas (ADR)

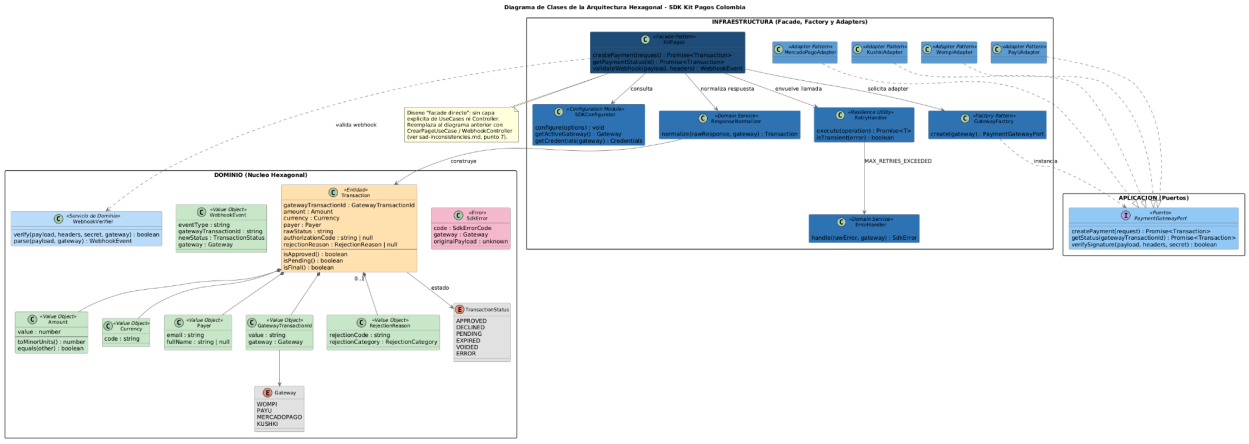


Figura 16. Diagrama de la arquitectura hexagonal

13.1. ADR-01: Adopción de Arquitectura Hexagonal para el núcleo del SDK

ASR asociado: ASR-01, Extensibilidad ante nuevas pasarelas de pago

Problema. El proyecto nace de un problema muy concreto: cuatro pasarelas de pago colombianas (Wompi, Rapyd, Mercado Pago y Kushki) exponen contratos completamente distintos entre sí, con diferencias en autenticación, formato de solicitud, nomenclatura de estados y mecanismos de firma. Si el SDK acoplara su lógica de negocio directamente a las particularidades de cada proveedor, cualquier cambio en una API externa, o la necesidad de incorporar una quinta pasarela en el futuro, obligaría a modificar el núcleo del sistema, incrementando el riesgo de introducir errores en funcionalidades que no tenían relación con el cambio.

Solución. Se organizó el SDK siguiendo el estilo de Arquitectura Hexagonal, también conocido como Puertos y Adaptadores, separando el código en tres capas independientes: dominio, aplicación e infraestructura. El dominio concentra las reglas de negocio y no conoce ninguna pasarela en particular; la comunicación con el exterior ocurre exclusivamente a través del puerto *PaymentGatewayPort*, definido en la capa de aplicación, mientras que cada proveedor se traduce mediante un adaptador independiente (*WompiAdapter, RapydAdapter, MercadoPagoAdapter y KushkiAdapter)* que implementa dicho puerto siguiendo el patrón GoF Adapter. Esta organización puede verse en el Diagrama de Componentes del SDK y en el Diagrama de Clases de la Arquitectura Hexagonal (Figura 16), donde se aprecia que ningún componente del dominio referencia directamente un tipo o estructura propia de Wompi, Rapyd, Mercado Pago o Kushki.

Consecuencias. Como consecuencia positiva, agregar una nueva pasarela se convierte en una tarea acotada a la creación de un nuevo adaptador, sin tocar el dominio, la fachada ni los adaptadores existentes, lo que además reduce directamente el acoplamiento (CBO) medido sobre el código cliente del SDK. Como consecuencia negativa, esta separación introduce una mayor cantidad de archivos y una curva de entendimiento inicial más alta para quien no esté familiarizado con el estilo hexagonal, en comparación con una arquitectura en capas tradicional donde el código estaría más concentrado aunque más acoplado.

13.2. ADR-02: Adopción de Arquitectura Hexagonal para el núcleo del SDK

ASR asociado: ASR-02, Cambio de pasarela activa sin modificar código cliente.

Problema. Un comercio que integra el SDK necesita poder cambiar de pasarela de pago, por ejemplo pasar de Wompi a Rapyd, sin que ese cambio implique reescribir su lógica de negocio ni recompilar partes del sistema que no

tienen relación directa con la pasarela. Si la selección del proveedor quedara dispersa entre distintos componentes, o si el desarrollador tuviera que instanciar manualmente el adaptador correspondiente, el cambio de pasarela dejaría de ser una operación de configuración y se convertiría en un cambio de código con riesgo de introducir errores.

Solución. Se introdujo el SDK Configurator como punto único de gestión de la configuración, responsable de mantener la pasarela activa, las credenciales por proveedor, el modo de entorno y los parámetros del mecanismo de reintento. A partir de esa configuración, el Gateway Factory, implementado bajo el patrón GoF Factory, recibe el valor del enum Gateway y retorna la instancia del adaptador correspondiente sin que el resto del sistema conozca ese proceso de instanciación. El Payment Facade, como único punto de entrada del SDK, consulta al Configurator y delega en el Factory antes de ejecutar cualquier operación, tal como se describe en el Diagrama de Componentes del SDK.

Consecuencias. La consecuencia positiva más directa es que cambiar de pasarela se reduce a modificar un parámetro de inicialización del SDK, sin que el desarrollador que lo consume necesite conocer ni modificar la lógica interna de selección de adaptadores. La consecuencia negativa es que el Gateway Factory sigue siendo el componente que debe actualizarse cada vez que se incorpora una nueva pasarela al enum Gateway, por lo que, aunque el acoplamiento se reduce drásticamente para el código cliente, no desaparece por completo dentro del propio SDK.

13.3. ADR-03: Adopción de Arquitectura Hexagonal para el núcleo del SDK

ASR asociado: ASR-03, Normalización uniforme de respuestas entre pasarelas

Problema. Cada pasarela devuelve el resultado de una transacción con su propio vocabulario: Wompi usa APPROVED, Rapyd usa el valor CLO acompañado de un campo booleano paid Mercado Pago responde en minúsculas con *approved*, y Kushki utiliza el término *APPROVAL* en lugar de *APPROVED* para un pago exitoso. Si el desarrollador que integra el SDK tuviera que interpretar directamente estos valores nativos, su código quedaría acoplado a los formatos de cada proveedor y expuesto a errores sutiles, como el caso documentado de Kushki, donde una comparación literal contra *APPROVED* clasificaría incorrectamente todos los pagos exitosos.

Solución. Se definió el Response Normalizer como servicio de aplicación encargado de traducir el estado nativo devuelto por cualquier adaptador hacia el enum canónico *TransactionStatus*, con los valores *APPROVED*, *DECLINED*, *PENDING*, *EXPIRED*, *VOIDED* y *ERROR*. Este servicio construye además la entidad de dominio *Transaction*, que expone métodos de negocio como *isApproved()* o *isFinal()*, de modo que el desarrollador razone sobre el resultado de un pago sin conocer el formato original de la pasarela. El valor nativo se conserva en el campo *rawStatus* para efectos de auditoría, siguiendo la tabla de equivalencias documentada en el lenguaje ubicuo del proyecto. El flujo completo, desde la respuesta nativa hasta la entidad normalizada, está representado en el Diagrama de Secuencia de Creación de Pago.

Consecuencias. Entre las consecuencias positivas está que el código cliente del SDK nunca necesita referenciar los tipos de respuesta de ninguna pasarela, lo que reduce su acoplamiento y hace que la lógica de negocio del comercio sea independiente del proveedor activo. Entre las consecuencias negativas está que la tabla de mapeo de estados debe mantenerse y ampliarse manualmente cada vez que se agrega una pasarela nueva o que un proveedor existente modifica su vocabulario de estados, concentrando en este único componente un riesgo de error que antes estaba distribuido entre varios adaptadores.

13.4. ADR-04: Verificación criptográfica de firmas para webhooks entrantes

Problema. Las notificaciones asíncronas de las pasarelas llegan como solicitudes HTTP hacia un endpoint del comercio, y nada impide, en principio, que un tercero malintencionado lance una solicitud fabricada que simule un pago aprobado si no existe un mecanismo de verificación. El reto adicional es que cada pasarela firma sus webhooks de forma distinta, Wompi mediante SHA-256 sobre una cadena de propiedades más un secreto, Rapyd mediante

HMAC-SHA256 sobre una cadena que incluye la URL pública configurada para el webhook, una marca de tiempo y las credenciales del comercio, codificando el resultado en Base64, y Mercado Pago mediante HMAC-SHA256 sobre encabezados y cuerpo.

Solución. Se definió el *Webhook Verifier* como servicio de aplicación con un único método *verify(payload, headers, secret, gateway)*, que internamente delega en la implementación de firma correspondiente a cada proveedor. Este componente se apoya en el mismo puerto de dominio para mantener el aislamiento respecto a las particularidades de cada pasarela, de forma consistente con la Arquitectura Hexagonal adoptada en el ADR-01, y su interacción con el resto del sistema se documenta en el Diagrama de Secuencia de Conciliación de Webhooks. Solo después de que la firma es validada exitosamente, el resultado se delega hacia la actualización del estado de la entidad Transaction.

Consecuencias. La consecuencia positiva es que el SDK ofrece una única función de verificación independiente del proveedor, evitando que el comercio implemente su propia lógica criptográfica y reduciendo la superficie de error ante ataques de suplantación de eventos. La consecuencia negativa es que la robustez de esta verificación depende directamente de la documentación pública de cada pasarela.

13.5. ADR-05: Reintentos con backoff exponencial ante fallas transitorias

ASR asociado: ASR-05, Recuperación ante fallas transitorias de red

Problema. Las integraciones con servicios externos están expuestas a fallos que no reflejan un problema de negocio, como timeouts de red o respuestas HTTP 5xx del lado de la pasarela. Si el SDK propagara inmediatamente cualquiera de estos errores hacia el desarrollador, una simple congestión momentánea en la infraestructura del proveedor terminaría reportándose como un fallo definitivo, obligando al comercio a implementar su propia lógica de reintentos de forma dispersa y potencialmente inconsistente entre pasarelas.

Solución. Se incorporó el Retry Handler como servicio de aplicación que intercepta los errores antes de que lleguen al Error Handler, distinguiendo entre errores transitorios candidatos a reintento, como timeouts o códigos 5xx, y errores definitivos que no se recuperarán con intentos adicionales, como un rechazo de pago o credenciales inválidas. La estrategia adoptada es un backoff configurable, con tres reintentos por defecto e intervalos de uno, dos y cuatro segundos. Si se agota el número máximo de intentos, el error se entrega al Error Handler con el código *MAX_RETRIES_EXCEEDED*.

Consecuencias. La consecuencia positiva es que el SDK absorbe automáticamente una clase completa de fallas de infraestructura que, de otro modo, degradarían la experiencia de integración sin ninguna relación con la lógica de negocio del comercio. La consecuencia negativa es que introduce latencia adicional en escenarios donde el reintento efectivamente no tendrá éxito, y que la clasificación entre error transitorio y definitivo depende de reglas fijas que podrían no cubrir todos los casos particulares de cada pasarela conforme estas evolucionen.

13.6. ADR-06: Motor de escenarios controlados en la API de Simulación

ASR asociado: ASR-06, Reproducción fiel de escenarios de prueba en el simulador

Problema. Validar el comportamiento del SDK frente a los distintos resultados posibles de un pago (aprobado, rechazado, fondos insuficientes, timeout o caída del servidor) requeriría, en un escenario real, provocar esas condiciones directamente sobre las pasarelas de producción o sandbox, lo cual no siempre es posible ni deseable durante la fase de evaluación académica del proyecto. Sin un mecanismo controlado, el equipo no tendría forma confiable ni repetible de ejercitar cada rama de la lógica del SDK, en particular el Response Normalizer y el Retry Handler.

Solución. Se construyó la API de Simulación sobre Fastify y TypeScript, aprovechando su soporte nativo de validación mediante JSON Schema para definir con precisión la forma de las solicitudes y respuestas de cada

escenario. El flujo se organizó como una tubería de procesamiento secuencial: el HTTP Router identifica la pasarela a partir de la URL e intercepta los encabezados *x-simulate-scenario y x-simulate-delay*, el *Scenario Execution* Engine decide cómo proceder según el escenario solicitado, y el Gateway Mock Factory, junto con sus fábricas internas por proveedor, construye un payload que replica con exactitud la estructura nativa de Wompi, Rapyd, Mercado Pago o Kushki según corresponda. Este diseño está documentado en el Diagrama de Componentes de la API de Simulación, y su efecto observable se ilustra en los Diagramas de Secuencia de Pago Exitoso, Pago Rechazado y Error de Red.

Consecuencias. La consecuencia positiva es que se puede probar de forma determinista y repetible cada camino relevante del SDK sin depender de la disponibilidad ni del comportamiento real de las cuatro pasarelas externas, lo cual resulta particularmente valioso dado el contexto de evaluación académica del proyecto. La consecuencia negativa es que la validez de las pruebas depende por completo de la fidelidad de los mocks frente a las respuestas reales de cada proveedor, de modo que si una pasarela modifica su formato de respuesta y esa actualización no se refleja en el Gateway Mock Factory, las pruebas seguirían pasando aunque el SDK ya no fuera compatible con el comportamiento real de dicha pasarela.

14. Riesgos técnicos

Los riesgos técnicos de Kit Pagos Colombia surgen principalmente de las consecuencias negativas ya identificadas en los ADR de este documento, como el desfase entre los mocks del simulador y las respuestas reales de las pasarelas, la implementación parcial de verificación de firmas para tres de los cuatro proveedores, o la clasificación de errores transitorios en el mecanismo de reintentos, junto con riesgos propios de la infraestructura de despliegue y del alcance limitado por el tiempo del proyecto académico. Dado que cada riesgo requiere documentar probabilidad, impacto, prioridad, plan de mitigación y plan de contingencia, presentar esta información en una tabla dentro del documento obligaría a comprimir el texto o reducir el tamaño de la fuente para que fuera legible. Por esta razón, al igual que con los ASR, se optó por adjuntar una hoja de cálculo con el detalle completo de los riesgos técnicos: Riesgos tecnicos kit pagos colombia.

15. Estructura del Sistema

Esta sección tiene como propósito que el desarrollador comprenda el detalle de implementación de cada clase que compone el sistema, complementando lo ya explicado a nivel conceptual en el Modelo de Dominio (sección 3) y en los Diagramas de Componentes (secciones 9.1 y 9.2). Aquí no se repite el propósito de cada componente, ya justificado en esas secciones, sino que se detallan sus atributos, métodos y asociaciones con el nivel de precisión necesario para guiar la implementación de las clases.

15.1 Núcleo del dominio

Transaction es la única clase con identidad propia del dominio y se construye a partir de los objetos de valor Amount, Currency, OrderReference, Payer y GatewayTransactionId, además de referenciar el enum TransactionStatus que indica su estado y, opcionalmente, una instancia de RejectionReason cuando ese estado es DECLINED. Incluye también el atributo opcional authorizationCode, fijado en el constructor cuando la pasarela lo provee. Expone los métodos isApproved(), isFinal() e isPending(), que permiten al código cliente razonar sobre el resultado de un pago sin necesidad de comparar directamente contra los valores del enum. Conserva además el campo rawStatus, que guarda el valor nativo devuelto por la pasarela antes de ser normalizado, con fines de auditoría.

A diferencia de lo que podría sugerir un diagrama con un método de actualización de estado, Transaction es inmutable: ninguno de sus atributos se modifica una vez construida la instancia. Cuando llega un webhook de conciliación con un cambio de estado sobre una transacción que ya existía como PENDING, el ResponseNormalizer no muta la instancia original, sino que construye una nueva instancia de Transaction con el estado actualizado a partir del evento recibido. Esta decisión es consistente con el tratamiento de Transaction como agregado inmutable dentro del dominio y evita efectos secundarios en referencias que otro componente pudiera conservar sobre la instancia previa.

Los objetos de valor que componen a Transaction deben implementarse como inmutables. Amount encapsula un único atributo numérico con hasta dos decimales y expone toMinorUnits() para que cada Adapter lo traduzca a la representación nativa de su pasarela, además de un método equals() para comparación por valor. Currency valida en su constructor que el código recibido tenga exactamente tres letras mayúsculas conforme a ISO 4217, con "COP" como valor por defecto. OrderReference valida únicamente que su valor no esté vacío, sin generar ni garantizar unicidad. Payer agrupa los datos de quien paga y valida en su constructor que el campo email esté presente, siendo el único atributo obligatorio. GatewayTransactionId combina el identificador nativo con el Gateway que lo originó, y se diferencia de OrderReference en que es la pasarela quien lo genera, no el comercio. RejectionReason combina el código nativo del rechazo (rejectionCode) con la categoría normalizada (rejectionCategory) tomada del enum RejectionCategory. ReturnUrlConfig expone el método resolveFor(status), que retorna la URL correspondiente según el resultado del pago, soportando tanto una URL única como URLs diferenciadas por resultado.

WebhookEvent es un objeto de valor inmutable del dominio que representa el evento notificado de forma asíncrona por una pasarela cuando ocurre un cambio de estado en una transacción. Encapsula el identificador de transacción nativo (gatewayTransactionId), el tipo de evento normalizado, la pasarela de origen (gateway), el estado resultante mapeado al dominio (status) y el payload nativo original (rawPayload). Es retornado por las operaciones de parsing de webhooks para permitir una manipulación tipada y uniforme de las notificaciones sin depender de las estructuras heterogéneas de cada proveedor.

WebhookVerifier se implementa como servicio de dominio sin estado propio. Expone dos métodos públicos: verify(payload, headers, secret, gateway): boolean, que valida la integridad criptográfica de la firma del mensaje, y parse(payload, gateway): WebhookEvent, que transforma la notificación entrante de una pasarela específica en una instancia normalizada de WebhookEvent para dar cumplimiento a RF-04. Internamente delega la verificación y extracción de atributos en la lógica correspondiente al Gateway recibido.

SdkError extiende la clase Error nativa de JavaScript y añade los atributos code (del tipo SdkErrorCode), gateway y originalPayload, este último tipado como unknown para forzar una verificación explícita antes de su uso. Los enums TransactionStatus, RejectionCategory, SdkErrorCode y Gateway no requieren métodos propios; su detalle de valores ya está documentado en la sección 3 y no se repite aquí.

15.2 SDK

PaymentFacade es la única clase que el desarrollador que consume el SDK instancia directamente. Mantiene una referencia a SdkConfigurator y a GatewayFactory, y expone los tres métodos públicos del SDK: createPayment(), getPaymentStatus() y validateWebhook(). Antes de ejecutar cualquier operación, consulta al Configurator para obtener la pasarela activa y sus credenciales, solicita al Factory la instancia del adaptador correspondiente, y envuelve la llamada resultante con RetryHandler.

SdkConfigurator centraliza el estado de configuración del SDK: la pasarela activa, un mapa de credenciales por proveedor, el entorno de ejecución (production, sandbox o simulation) y los parámetros del mecanismo de reintento. Expone configure(options) para inicializar esos valores y métodos de acceso puntual como getActiveGateway() y

getCredentials(gateway). GatewayFactory no mantiene estado; su único método, create(gateway), recibe un valor del enum Gateway y retorna la instancia del Adapter correspondiente, ocultando ese proceso de instanciación del resto del sistema.

Los cuatro Adapters (WompiAdapter, RapydAdapter, MercadoPagoAdapter y KushkiAdapter) implementan la interfaz PaymentGatewayPort, que define los métodos createPayment(), getStatus() y verifySignature(). Cada Adapter mantiene su propio cliente HTTP y sus credenciales, y traduce internamente esos tres métodos hacia las convenciones específicas de su proveedor: autenticación por encabezados HTTP (access_key, salt, timestamp y firma) en Rapyd, Bearer en Mercado Pago, el header Private-Merchant-Id en Kushki, y el algoritmo de firma correspondiente en cada caso (SHA-256 para Wompi, HMAC-SHA256 con codificación Base64 para Rapyd, HMAC-SHA256 para Mercado Pago y Kushki). Ninguno de los cuatro expone métodos adicionales a los definidos por el puerto, para mantener la sustituibilidad entre ellos.

ResponseNormalizer mantiene internamente la tabla de equivalencias entre los estados nativos de cada pasarela y el enum TransactionStatus. Su método normalize(rawResponse, gateway) toma la respuesta cruda de cualquier Adapter y construye la entidad Transaction correspondiente, siendo invocado por PaymentFacade inmediatamente después de cada llamada exitosa a un Adapter.

RetryHandler envuelve las llamadas que PaymentFacade hace a los Adapters mediante su método execute(operation), que ejecuta la operación recibida y, ante una falla, consulta isTransient(error) para decidir si corresponde reintentar. Mantiene como atributos el número máximo de intentos (tres por defecto) y los intervalos de espera entre ellos (uno, dos y cuatro segundos por defecto). Si se agotan los intentos sin éxito, delega el error a ErrorHandler con el código MAX_RETRIES_EXCEEDED.

ErrorHandler mantiene un mapa de errores nativos hacia SdkErrorCode por cada Gateway, y expone el método handle(rawError, gateway), que traduce cualquier error recibido de un Adapter o del RetryHandler hacia una instancia de SdkError, siendo este el único tipo de error que finalmente se propaga al código cliente del SDK.

15.3 API de Simulación

HttpRouterMiddleware, implementado como plugin de Fastify, es responsable de dos tareas antes de delegar el procesamiento: identificar la pasarela de destino a partir de la estructura de la URL mediante identifyGateway(url), y extraer los headers de control de la solicitud mediante extractHeaders(req), que retorna el escenario solicitado (x-simulate-scenario) y la latencia a introducir (x-simulate-delay).

ScenarioExecutionEngine concentra la lógica de decisión del simulador en un único método, execute(scenario, gateway). Para los escenarios de negocio (aprobado, rechazado, fondos insuficientes) delega la construcción del payload a GatewayMockFactory; para el escenario de timeout, introduce la latencia recibida antes de responder; para error de red, retorna directamente una respuesta HTTP 500 o 502 sin invocar a la fábrica.

GatewayMockFactory expone el método build(scenario, gateway) y se compone internamente de cuatro fábricas específicas por proveedor: WompiMockFactory, RapydMockFactory, MercadoPagoMockFactory y KushkiMockFactory. Cada una de estas cuatro clases internas conoce el vocabulario de campos y valores propio de su pasarela y expone su propio método build(scenario), sin ser accesibles directamente desde fuera de GatewayMockFactory.

SignatureGenerator expone el método sign(payload, gateway, secret) y es invocado exclusivamente por WebhookTriggerEndpoint, nunca de forma automática. WebhookTriggerEndpoint implementa el endpoint POST /v1/sim/webhooks/trigger mediante su método trigger(targetUrl, eventType, gateway), que solicita la firma

correspondiente a SignatureGenerator antes de despachar el webhook sintético hacia la URL indicada por quien lo invoca.

OpenApiDocumentationProvider, implementado mediante el plugin Swagger de Fastify, no expone métodos propios de negocio: genera automáticamente la especificación OpenAPI 3.0 a partir de las definiciones de rutas registradas en HttpRouterMiddleware, sirviéndola en el endpoint /docs.

16. Glosario

- **Adapter (Adaptador).** Patrón de diseño GoF que traduce la interfaz de un componente hacia otra que el sistema espera. Cada pasarela de pago tiene su propio Adapter (WompiAdapter, RapydAdapter, etc.) que traduce el contrato genérico del SDK hacia las convenciones específicas de esa pasarela.
- **API REST.** Interfaz de programación que expone operaciones a través de solicitudes HTTP (GET, POST, etc.) siguiendo el estilo arquitectónico REST.
- **API de Simulación.** Servicio web desarrollado en este proyecto que replica el comportamiento de las cuatro pasarelas de pago soportadas, permitiendo probar integraciones sin usar credenciales reales.
- **ASR (Architecturally Significant Requirement).** Requisito Arquitectónicamente Significativo: requisito funcional o no funcional que tiene un impacto directo y relevante sobre las decisiones de diseño de la arquitectura del sistema.
- **Backoff exponencial.** Estrategia de reintento en la que el tiempo de espera entre intentos sucesivos aumenta de forma exponencial (por ejemplo, 1s, 2s, 4s), utilizada para evitar sobrecargar un servicio que está fallando temporalmente.
- **CBO (Coupling Between Objects).** Métrica del conjunto de Chidamber y Kemerer (1994) que mide el número de clases a las que una clase está acoplada. Se usa en este proyecto para comparar el acoplamiento resultante de integrar pasarelas con y sin el framework.
- **Credenciales de sandbox.** Claves o tokens de prueba proporcionados por una pasarela de pago para realizar transacciones simuladas sin mover dinero real.
- **CK (Métricas de Chidamber y Kemerer).** Conjunto de métricas de calidad de código orientado a objetos (entre ellas WMC, CBO y RFC), propuesto por Chidamber y Kemerer en 1994, usado en este proyecto para evaluar objetivamente el impacto del framework.
- **DDD (Domain-Driven Design).** Enfoque de diseño de software que organiza el sistema alrededor de un modelo del dominio del negocio, distinguiendo entre Entidades, Objetos de Valor y Servicios de Dominio, tal como se aplica en el modelo de dominio de este documento (sección 3).
- **Entidad (DDD).** En Domain-Driven Design, un objeto que tiene una identidad propia que persiste a lo largo del tiempo, independientemente de sus atributos. Transaction es la única entidad del dominio en este proyecto.
- **Facade (Fachada).** Patrón de diseño GoF que provee una interfaz simplificada y unificada hacia un subsistema más complejo. PaymentFacade es el único punto de entrada que el desarrollador consume del SDK.
- **Factory (Fábrica).** Patrón de diseño GoF que encapsula la lógica de creación de objetos, ocultando el proceso de instanciación del código cliente. GatewayFactory retorna el Adapter correspondiente a la pasarela activa.
- **Fastify.** Framework web para Node.js orientado al alto rendimiento, utilizado para implementar la API de Simulación.
- **FURPS+.** Modelo de clasificación de atributos de calidad de software (Functionality, Usability, Reliability, Performance, Supportability, más restricciones adicionales), usado en este documento para organizar los atributos de calidad de la arquitectura.
- **Gateway (Pasarela de pago).** Servicio externo que procesa transacciones de pago en nombre de un comercio. Las cuatro pasarelas soportadas en este proyecto son Wompi, Rapyd, Mercado Pago y Kushki.
- **HMAC-SHA256.** Algoritmo de código de autenticación de mensajes basado en la función hash SHA-256, usado por Mercado Pago y Kushki para firmar sus webhooks y permitir verificar que no fueron alterados.
- **Hexagonal, Arquitectura (Puertos y Adaptadores).** Estilo arquitectónico que separa el núcleo de negocio de un sistema de sus dependencias externas mediante puertos (interfaces) y adaptadores (implementaciones concretas), adoptado como base del SDK en el ADR-01.
- **HTTPS.** Protocolo HTTP sobre una capa de cifrado TLS, usado obligatoriamente en toda comunicación del framework con las pasarelas de pago y la API de Simulación.

- **Idempotencia.** Propiedad de una operación que produce el mismo resultado sin efectos adicionales, sin importar cuántas veces se ejecute. Es relevante para el manejo de reintentos y de webhooks duplicados, un escenario que cualquier pasarela puede generar y que el Webhook Verifier debe manejar sin asumir que cada notificación llega exactamente una vez.
- **JSON Schema.** Especificación para describir y validar la estructura de documentos JSON, utilizada por Fastify en la API de Simulación para validar solicitudes y respuestas de cada escenario.
- **Mock (payload simulado).** Objeto o respuesta construida artificialmente para replicar, con fines de prueba, el comportamiento de un componente real; en este caso, las respuestas nativas de cada pasarela de pago.
- **npm.** Gestor de paquetes de Node.js utilizado para distribuir e instalar el SDK del framework mediante un único comando.
- **Objeto de Valor (Value Object).** En Domain-Driven Design, un objeto sin identidad propia que se define únicamente por sus atributos y que debería ser inmutable. Amount, Currency y Payer, entre otros, son objetos de valor en este proyecto.
- **OpenAPI 3.0.** Especificación estándar para describir APIs REST de forma legible tanto por humanos como por máquinas, expuesta públicamente por la API de Simulación en el endpoint /docs.
- **PaaS (Platform as a Service).** Modelo de servicio en la nube que provee una plataforma completa (runtime, infraestructura) para desplegar aplicaciones sin gestionar servidores directamente. La API de Simulación se despliega sobre una plataforma PaaS (Render).
- **PCI DSS (Payment Card Industry Data Security Standard).** Estándar de seguridad para el manejo de datos de tarjetas de pago. El framework delega su cumplimiento a las pasarelas integradas, ya que no procesa ni almacena datos de tarjeta directamente.
- **Puerto (Port).** En la Arquitectura Hexagonal, una interfaz definida por el núcleo del sistema que especifica qué operaciones necesita del exterior, sin conocer cómo se implementan. PaymentGatewayPort es el puerto que todos los Adapters de pasarela deben implementar.
- **RFC (Response For a Class).** Métrica del conjunto de Chidamber y Kemerer que mide el número de métodos que pueden ejecutarse en respuesta a un mensaje recibido por una clase, incluyendo los suyos propios y los que invoca.
- **RF (Requisito Funcional).** Requisito que describe una funcionalidad concreta que el sistema debe realizar, numerado en este proyecto como RF-01, RF-02, etc.
- **Sandbox.** Entorno de pruebas aislado que provee una pasarela de pago para simular transacciones sin procesar dinero real.
- **SDK (Software Development Kit).** Conjunto de herramientas, en este caso una biblioteca de código distribuida como paquete npm, que un desarrollador integra en su aplicación para consumir las capacidades de pago del framework.
- **Servicio de Dominio.** En Domain-Driven Design, una operación de negocio que no pertenece naturalmente a ninguna Entidad ni Objeto de Valor por no tener datos propios que persistir. WebhookVerifier es un servicio de dominio en este proyecto.
- **SHA-256.** Función hash criptográfica utilizada, entre otros usos, por Wompi para firmar sus webhooks.
- **TypeScript.** Superconjunto tipado de JavaScript, lenguaje en el que está implementado el SDK y la API de Simulación de este proyecto.
- **Webhook.** Mecanismo mediante el cual un sistema notifica a otro, de forma asíncrona, sobre la ocurrencia de un evento, enviando una solicitud HTTP POST hacia una URL previamente configurada. Las pasarelas de pago lo usan para notificar cambios de estado en una transacción.
- **WMC (Weighted Methods per Class).** Métrica del conjunto de Chidamber y Kemerer que mide la complejidad de una clase como la suma de la complejidad de sus métodos.

17. Referencias

Referencias bibliográficas en formato APA o IEEE que hayan sido usadas en el documento